

NAUKRI_GURU: AN AI-POWERED PLATFORM FOR AUTOMATED JOB APPLICATION, LIFECYCLE TRACKING, AND RECRUITER OUTREACH ON LINKEDIN

A Project Report Submitted
in Partial Fulfilment of the Requirements
for the Degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

by

Manvendra Pratap Singh (Roll No. CS22B1054)



ಭಾರತೀಯ ಮಾಹಿತಿ ತಂತ್ರಜ್ಞಾನ ಸಂಸ್ಥೆ ರಾಯಚೂರು
भारतीय सूचना प्रौद्योगिकी संस्थान रायचूर
Indian Institute of Information Technology Raichur

to

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INDIAN INSTITUTE OF INFORMATION TECHNOLOGY
RAICHUR-584135, INDIA

May 2026

DECLARATION

I **MANVENDRA PRATAP SINGH** (Roll Number: CS22B1054) hereby declare that, this report entitled “*Naukri_Guru: An AI-Powered Platform for Automated Job Application, Lifecycle Tracking, and Recruiter Outreach on LinkedIn*” submitted to Indian Institute of Information Technology Raichur towards partial requirement of Bachelor of Technology in Computer Science and Engineering is an original work carried out by me under the supervision of Mentor, Dr. Neha Agarwal, Assistant Professor and has not formed the basis for the award of any degree or diploma, in this or any other institution or university. I have sincerely tried to uphold the academic ethics and honesty. Whenever an external information or statement or result is used then, that have been duly acknowledged and cited.

Raichur – 584135
May 2026

Manvendra Pratap Singh
Roll No. CS22B1054

CERTIFICATE

This is to certify that the work contained in this project report entitled “*Naukri_Guru: An AI-Powered Platform for Automated Job Application, Lifecycle Tracking, and Recruiter Outreach on LinkedIn*” submitted by **Manvendra Pratap Singh** (Roll No: CS22B1054) to the Indian Institute of Information Technology Raichur towards partial requirement of Bachelor of Technology in Computer Science and Engineering has been carried out by him under my supervision and that it has not been submitted elsewhere for the award of any degree.

Raichur – 584135
May 2026

(Dr. Neha Agarwal, Assistant
Professor)
Project Supervisor

ABSTRACT

Job hunting in the contemporary digital landscape demands considerable manual effort, as candidates routinely submit hundreds of applications through online portals while simultaneously tracking outcomes and engaging with recruiters. This report introduces **Naukri_Guru**, an intelligent job automation platform engineered to streamline the entire LinkedIn application lifecycle through browser automation, artificial intelligence, and structured data management. The platform operates on a five-layer modular architecture comprising a user interface layer, an orchestration engine, an intelligence layer with support for multiple AI providers, a stealth browser automation layer, and a SQLite-backed persistence layer.

The system automates job discovery, role suitability evaluation through a heuristic confidence scoring engine, and application submission via LinkedIn’s Easy Apply mechanism. A three-tier question answering cascade—combining statically configured personal data, a persistent memory store, and generative AI providers (Google Gemini, OpenAI, and DeepSeek) protected by a safety guard function—enables fully autonomous form completion. Beyond LinkedIn, the platform incorporates an Indeed job scraper that discovers and archives relevant positions for manual review. Post-application lifecycle management is achieved through Gmail IMAP synchronization, which classifies recruiter emails and matches them to stored applications, while a cold email outreach pipeline facilitates proactive recruiter engagement with AI-generated personalized messages.

Operational evaluation across twenty-three automation sessions demonstrates that the platform processes over eight hundred job listings, achieves an application success rate exceeding eighty-seven percent after server-side verification, and reduces per-application time by approximately ninety percent compared to manual effort. The memory-based question answering system converges toward near-autonomous operation within eight to nine sessions, progressively eliminating the need for human intervention in form completion.

Contents

Declaration	ii
Certificate	iii
Abstract	iv
List of Figures	viii
List of Tables	viii
1 Introduction	1
1.1 Problem Statement	2
1.2 Project Significance	2
1.3 Scope of Work	3
1.4 Objectives	4
1.5 Novelty	5
2 Literature Survey	7
2.1 Existing Approaches – Job Application Automation Tools	7
2.1.1 LazyApply	7
2.1.2 EasyApplyBot (Open-Source)	8
2.1.3 JobScan	8
2.1.4 Differentiation of Our Approach	8
2.2 Existing Approaches – Browser Automation and Anti-Detection	9
2.2.1 Selenium WebDriver	9
2.2.2 Puppeteer and Playwright	10
2.2.3 Undetected ChromeDriver	10
2.2.4 Differentiation of Our Approach	10
2.3 Existing Approaches – AI-Assisted Form Filling and NLP	11
2.3.1 GPT-Based Auto-Fill Systems	11
2.3.2 Resume Parsing and Information Extraction	11
2.3.3 Differentiation of Our Approach	12

3	Methodology	13
3.1	Problem Restatement	13
3.2	Overall Approach	14
3.3	System Block Diagram	14
3.4	Architecture	16
3.5	Module One: Stealth Browser Automation Engine	17
3.6	Module Two: Job Discovery and Intelligent Filtering	19
3.7	Module Three: Intelligent Question Answering	21
3.8	Module Four: Application Lifecycle and Data Persistence	24
3.9	Module Five: Indeed Job Scraper	26
4	Challenges and Solutions	28
5	Results and Discussion	31
5.1	Application Automation Results	31
5.1.1	Session Execution Summary	31
5.1.2	Filtering Funnel Analysis	32
5.1.3	Time Savings Estimation	33
5.2	Gmail Lifecycle Sync Results	33
5.2.1	Email Classification Accuracy	34
5.2.2	Cold Email Outreach Results	35
5.3	Indeed Scraper Results	36
5.3.1	Data Export and Reporting	36
5.3.2	Confidence Score Distribution	37
5.4	Discussion	38
6	Evaluation, Benchmarking, and Comparative Analysis	40
6.1	Evaluation Methodology	40
6.1.1	Runtime Validation Strategy	40
6.1.2	Manual and Automated Testing	41
6.1.3	Controlled QA Runs	42
6.2	Benchmarking Against Existing Approaches	42
6.2.1	Compared Approaches	42
6.2.2	Feature-Level Comparison	43
6.2.3	Quantitative Efficiency Comparison	44
6.3	Experimental Results	44
6.3.1	Application Submission Results	44
6.3.2	Cold Email Execution Results	45

6.3.3	Question Answering Accuracy Over Time	46
6.3.4	Runtime Recovery and Stability Observations	46
6.3.5	Export Synchronization Consistency	47
6.4	Performance Metrics	48
6.5	System Stability Analysis	49
6.5.1	Selenium Session Health	49
6.5.2	Queue-Based Cold Email Processing	49
6.5.3	Atomic CSV Writes	49
6.5.4	SQLite Synchronization	49
6.6	Novel Contributions	50
6.7	Limitations and Future Improvements	51
6.7.1	Current Limitations	51
6.7.2	Future Improvements	51
7	Tools, Technologies, and Parameters	53
7.1	Tools and Technologies	53
7.2	Key Parameters and Settings	54
8	Conclusion and Future Scope	56
	Bibliography	58

List of Figures

3.1	System Block Diagram – Five-Layer Architecture of Naukri_Guru	15
3.2	Software Architecture – Component Interaction Diagram	16
3.3	Module One – Stealth Browser Initialization Process Flow	18
3.4	Module Two – Job Discovery and Intelligent Filtering Process Flow	20
3.5	Module Three – Intelligent Question Answering Process Flow	22
3.6	Module Four – Application Lifecycle and Data Persistence Flow	24
5.1	Console Output of a Typical LinkedIn Automation Session	32
5.2	Gmail IMAP Lifecycle Sync – Console Output Showing Status Updates	34
5.3	Cold Email Outreach Pipeline – Console Output	36
5.4	Exported Excel Report Showing Multi-Source Application Tracking Data	37
5.5	Distribution of Apply Confidence Scores Across Processed Listings	38

List of Tables

2.1	Comparison of Job Application Automation Tools	9
2.2	Comparison of Browser Automation Frameworks	11
2.3	Comparison of AI-Assisted Form Filling Approaches	12
4.1	Major Implementation Challenges and Their Solutions	29
5.1	Time Savings: Manual vs. Automated Application	33
5.2	Gmail Classification Results Over 20-Day Lookback Period	35
6.1	Feature-Level Benchmarking: Naukri_Guru vs. Existing Approaches . . .	43
6.2	Quantitative Efficiency: Standardised 7-Application Workload	44
6.3	Aggregate Application Results Across 23 Evaluated Sessions	45
6.4	Cold Email Pipeline Results (Test Mode with Whitelisted Recipients) . .	45
6.5	Question Answer Source Distribution Across Sessions (Chronological) . .	46
6.6	Runtime Recovery Events Across 23 Sessions	47
6.7	Consolidated Performance Metrics	48
7.1	Important Parameters Used in Implementation	55

Chapter 1

Introduction

Over the past decade the manner in which job seekers and employers connect has shifted decisively toward online platforms. Professional networking services, foremost among them LinkedIn, have emerged as the dominant channel through which millions of positions are advertised and filled each year. LinkedIn’s own disclosures indicate a registered base exceeding nine hundred million members across more than two hundred countries, with tens of millions of companies maintaining active recruitment presences on the platform [1]. For candidates entering competitive fields—particularly those in software engineering, data science, and allied technology disciplines—the sheer volume of available openings creates a paradox: while opportunities are abundant, the administrative burden of identifying suitable roles, completing application forms, and monitoring outcomes consumes a disproportionate share of the job seeker’s time and cognitive energy.

This project addresses the inefficiency inherent in repetitive job application workflows by developing **Naukri_Guru**, an AI-powered automation platform that converts LinkedIn’s “Easy Apply” process into a largely autonomous pipeline. Complementing this core capability, the system also integrates an Indeed job scraper that discovers and archives relevant listings from a second major job portal, thereby broadening the candidate’s visibility across the employment landscape. Unlike simplistic macro recorders or rigid browser scripts, Naukri_Guru employs a layered software design that incorporates intelligent decision-making, persistent learning, structured data storage, email-based life-cycle tracking, and recruiter outreach—collectively transforming a tedious manual process into an efficient, data-driven operation.

The subsequent sections of this chapter describe the specific problems that motivated the development of Naukri_Guru, articulate the significance of the proposed solution, define the boundaries of the implementation, and highlight the distinctive contributions of this work relative to existing tools.

1.1 Problem Statement

Candidates pursuing technology roles encounter a set of interrelated obstacles that collectively render the application process slow, repetitive, and prone to errors:

- **Application Volume Demands:** An engineering graduate actively searching for employment may need to submit between two hundred and five hundred applications before securing an acceptable offer. Even through LinkedIn’s streamlined Easy Apply mechanism, each submission requires reading the job description, interacting with multi-step modal forms, uploading a resume, and answering screening questions—a sequence that consumes three to five minutes per application when performed manually.
- **Redundant Question Answering:** The screening questions embedded within LinkedIn’s Easy Apply modals overlap substantially across employers. Queries regarding years of professional experience, visa sponsorship eligibility, salary expectations, notice period, and location preferences recur in nearly every application, forcing candidates to provide identical or near-identical responses hundreds of times.
- **Absence of Intelligent Pre-Screening:** Although LinkedIn offers basic search filters, it does not prevent candidates from encountering and spending time on fundamentally mismatched listings—senior positions appearing in entry-level searches, roles demanding security clearances that the candidate cannot obtain, or openings at companies the candidate has already decided to avoid.
- **Fragmented Outcome Tracking:** Once an application has been submitted, there exists no centralized mechanism for monitoring which companies have responded, which applications have been silently passed over, and which have progressed to interview stages. Recruiter communications arrive through email, LinkedIn notifications, and third-party applicant tracking systems, requiring tedious manual consolidation.

1.2 Project Significance

The value of this project stems from its comprehensive approach to automating the job application lifecycle, extending well beyond what existing tools offer individually:

- **Dramatic Time Reduction:** Automated application submission compresses the per-application effort from three to five minutes of manual interaction to roughly fifteen to twenty seconds of machine-driven execution. Across a session involving one hundred candidate listings and five to seven actual submissions, this translates to a cumulative saving exceeding thirty minutes.
- **Heuristic Suitability Assessment:** Before committing to an application, the platform’s confidence scoring engine evaluates each job description against the candidate’s profile. The algorithm considers keyword relevance, seniority indicators, degree requirements, and experience thresholds, assigning a quantitative suitability score on a zero-to-one-hundred scale.
- **Adaptive Learning Through Memory:** The question answering module maintains a persistent JSON-based memory store that records previously provided answers. Over successive sessions, the fraction of questions requiring either AI assistance or human input diminishes steadily, and the system approaches fully autonomous form completion within approximately five sessions.
- **Multi-Portal Job Discovery:** In addition to LinkedIn, the Indeed scraper module broadens the candidate’s search surface by collecting and archiving relevant job listings from Indeed, presenting them in a structured spreadsheet format with colour-coded source attribution for manual review and follow-up.
- **Unified Lifecycle Visibility:** Gmail IMAP synchronization enables the platform to automatically detect and classify recruiter responses—including interview invitations, online assessment links, rejections, and offers—and update the application database accordingly, providing a consolidated view of the candidate’s entire job search trajectory.

1.3 Scope of Work

The implementation encompasses the following major components:

- **Stealth Browser Automation Engine:** A Chrome-based automation layer utilizing undetected-chromedriver and Selenium WebDriver, operating within an isolated browser profile to circumvent LinkedIn’s anti-bot defences. A Selenium Manager fallback guarantees driver resolution even when the primary stealth library encounters compatibility issues.

- **Job Discovery and Intelligent Filtering Pipeline:** A configurable search and filter mechanism that discovers LinkedIn job listings based on keywords, location, experience level, and additional parameters, then subjects each listing to a multi-factor filtering funnel encompassing blacklist checking, confidence scoring, seniority detection, and bad-word filtering.
- **AI-Assisted Question Answering Module:** A cascading answer resolution system that sequentially consults static configuration mappings, a persistent JSON memory cache, and one of three supported AI providers (Google Gemini, OpenAI, DeepSeek), with a dedicated safety guard function that intercepts and overrides AI-generated responses for personally sensitive question categories.
- **Indeed Job Scraper:** A collect-only scraping module that navigates Indeed's search interface, extracts job card metadata (title, company, location, URL), applies relevance filtering through configurable bad-word lists, and persists results to a unified job store with source attribution.
- **Application Lifecycle and Data Persistence Layer:** A SQLite-backed storage architecture with tables for applications, recruiters, lifecycle events, cold emails, and failure logs, complemented by CSV and XLSX export capabilities with colour-coded formatting and Gmail IMAP synchronization for automated status updates.
- **Cold Email Outreach Pipeline:** An automated recruiter engagement system that discovers recruiter contact details, generates personalised cold emails using AI, validates email trustworthiness through a multi-signal scoring mechanism, and dispatches messages via authenticated SMTP with rate limiting and duplicate prevention.

The platform targets the Windows operating system and is implemented in Python 3.10+. The modular architecture facilitates future extension to additional job portals beyond the currently supported LinkedIn and Indeed integrations.

1.4 Objectives

The principal objectives of this project are:

1. To architect and implement a stealth browser automation engine capable of navigating LinkedIn's dynamically rendered job search interface and Easy Apply modals without triggering anti-bot detection.

2. To develop a confidence scoring algorithm that quantitatively assesses the suitability of each discovered job listing relative to the candidate’s profile, producing a score on a zero-to-one-hundred scale.
3. To construct a multi-tier question answering system that combines static configuration, persistent memory, and generative AI to autonomously fill LinkedIn screening questions with contextually correct responses.
4. To build an Indeed job scraper that discovers, filters, and archives relevant job listings from Indeed for manual review, with source-attributed persistence in a unified job store.
5. To implement a structured SQLite persistence layer that records every application attempt, failure event, and lifecycle transition with complete audit trails and deduplication guarantees.
6. To integrate Gmail IMAP synchronization that classifies recruiter emails by application status and matches them to stored applications using multi-signal evidence scoring.
7. To engineer a cold email outreach pipeline with AI-generated personalised content, recruiter email trust validation, and SMTP delivery with sender reputation safeguards.

1.5 Novelty

Although several open-source and commercial LinkedIn automation tools exist, Naukri_Guru contributes a number of distinctive elements that set it apart from prior work:

- **Multi-Provider AI with Safety Guard Architecture:** Rather than depending on a single language model, the platform supports three AI providers through a unified interface. A dedicated guard function (`guard_ai_answer`) intercepts every AI response and overrides it with verified personal data whenever the question matches a safety-critical pattern—such as queries about graduation year, salary, or contact number—thereby preventing the AI from fabricating sensitive personal details.
- **Persistent Memory with Skill-Specific Experience Routing:** The memory module differentiates between general questions and skill-specific experience queries

(for instance, “How many years of React experience do you have?”) through a keyword detection mechanism, routing each to specialised handlers that maintain per-skill answer caches independent of the general memory store.

- **Post-Submission Verification Protocol:** Instead of assuming success after clicking the submit button, the system inspects the resulting DOM for LinkedIn’s confirmation markers (such as “application submitted” text or the presence of a “Done” button) and refuses to persist the application record unless positive verification is obtained, thereby eliminating phantom entries from the tracking database.
- **Dual-Portal Job Discovery:** The integration of an Indeed scraper alongside the LinkedIn automation engine provides cross-platform job visibility within a single tool, with a unified job store (`job_store.py`) that deduplicates across sources and produces colour-coded XLSX reports for streamlined manual review.
- **Runtime Batch Traceability:** Every automation session generates a unique `runtime_batch_id` that tags all database records, lifecycle events, and cold email dispatches produced during that session, enabling per-session audit trails and retrospective analysis.

The convergence of these capabilities produces an automation platform that functions not as a mechanical click-replay bot, but as an intelligent agent capable of adapting to diverse application forms, learning from user interactions, operating across multiple job portals, and maintaining rigorous data integrity throughout its operational lifecycle.

Chapter 2

Literature Survey

This chapter surveys the existing body of work across three domains that underpin the Naukri_Guru platform: job application automation tools, browser automation and anti-detection technologies, and AI-assisted form filling systems. For each domain, representative approaches are examined, their strengths and weaknesses are identified, and the manner in which Naukri_Guru extends or departs from them is articulated. The objective is to situate the project within the broader landscape of employment technology and intelligent web automation.

2.1 Existing Approaches – Job Application Automation Tools

The expansion of web-based recruitment has given rise to a range of tools designed to alleviate the manual labour associated with job applications. These solutions span commercial subscription services, open-source automation scripts, and resume optimization utilities.

2.1.1 LazyApply

LazyApply is a subscription-based automation service supporting several job platforms, including LinkedIn, Indeed, and ZipRecruiter [2]. It provides a browser extension that pre-populates profile data and submits applications on the user’s behalf. Despite its multi-platform reach, LazyApply stores all user data on remote servers, relies on template-based question answering without generative AI fallback, and provides no local persistence or post-application lifecycle tracking. Its subscription pricing model also creates an ongoing cost barrier for individual users.

2.1.2 EasyApplyBot (Open-Source)

EasyApplyBot is a community-maintained Python project built on Selenium that automates LinkedIn’s Easy Apply workflow [3]. The tool supports basic search configuration and fills form fields using hardcoded values. However, it lacks several capabilities that limit its practical utility: there is no persistent memory for previously answered questions, no heuristic scoring to filter unsuitable listings, no post-application tracking or lifecycle management, and no stealth measures beyond standard Selenium, leaving it susceptible to LinkedIn’s bot-detection heuristics.

2.1.3 JobScan

JobScan is a commercial resume analysis tool that compares a candidate’s resume against individual job descriptions and generates a compatibility score [4]. It excels at keyword analysis and Applicant Tracking System (ATS) compatibility assessment. However, it operates strictly as a pre-application advisory service and does not automate any part of the submission, tracking, or outreach workflow.

2.1.4 Differentiation of Our Approach

Naukri_Guru distinguishes itself from these tools across several dimensions:

- In contrast to LazyApply, all data in Naukri_Guru remains on the user’s local machine within a SQLite database, ensuring privacy and eliminating subscription costs.
- Unlike EasyApplyBot, this system implements a three-tier answer resolution cascade (configuration, memory, and AI with guard), heuristic confidence scoring, post-submission verification, and dual-portal job discovery through the Indeed scraper.
- While JobScan provides pre-application advice, Naukri_Guru automates the entire lifecycle from discovery through submission to recruiter engagement.

Table 2.1: Comparison of Job Application Automation Tools

Tool	Type	Year	Key Features	Limitations
LazyApply [2]	Commercial SaaS	2021	Multi-platform, browser extension, cloud profile	No local storage, subscription cost, no AI Q&A
EasyApplyBot [3]	Open-source	2022	Python/Selenium, Easy Apply automation	No memory, no scoring, no anti-detection, no lifecycle
JobScan [4]	Commercial SaaS	2018	Resume-JD matching, ATS optimization	Advisory only, no submission automation
Naukri_Guru	Open-source	2026	AI Q&A with guard, confidence scoring, Gmail sync, cold email, Indeed scraper, SQLite persistence	LinkedIn-primary, Windows-optimised

2.2 Existing Approaches – Browser Automation and Anti-Detection

Browser automation frameworks constitute the technical foundation upon which web-based application tools are constructed. The principal challenge extends beyond driving browser interactions to doing so without being identified as automated traffic by the target platform.

2.2.1 Selenium WebDriver

Selenium WebDriver remains the most widely adopted framework for programmatic browser control, offering a language-agnostic API compatible with Chrome, Firefox, Edge, and Safari through platform-specific driver binaries [5]. Its maturity and extensive documentation make it a natural starting point for automation projects. However, unmodified Selenium usage exposes detectable fingerprints—most notably the `navigator.webdriver` JavaScript property being set to `true`—which sophisticated platforms exploit to identify and block automated sessions.

2.2.2 Puppeteer and Playwright

Puppeteer, maintained by Google, and Playwright, maintained by Microsoft, are Node.js-based automation frameworks that communicate with browsers via the Chrome DevTools Protocol [6]. They provide superior performance for JavaScript-intensive single-page applications and offer built-in stealth capabilities. Their primary drawback for Python-centric projects is the Node.js runtime requirement, which introduces a language ecosystem barrier. Moreover, their stealth features are designed for general-purpose evasion rather than being tuned for the specific detection mechanisms employed by LinkedIn.

2.2.3 Undetected ChromeDriver

The `undetected-chromedriver` library is a Python package that patches the standard ChromeDriver binary to eliminate known automation indicators [7]. It modifies browser fingerprints, suppresses the `webdriver` flag, and randomises certain browser properties to simulate organic user traffic. This library is specifically designed for scenarios where standard Selenium triggers anti-bot protections, making it well-suited for LinkedIn automation.

2.2.4 Differentiation of Our Approach

Naukri_Guru's browser architecture combines multiple anti-detection strategies in a fault-tolerant configuration:

- **Primary Path:** The system employs `undetected-chromedriver` with automatic detection of the installed Chrome version, ensuring driver-browser version alignment and eliminating the common ChromeDriver mismatch failure.
- **Fallback Path:** If stealth startup fails, the platform automatically falls back to Selenium Manager for driver resolution, maintaining operational continuity regardless of the stealth library's compatibility state.
- **Profile Isolation:** All automation executes within a dedicated directory (`C:\Naukri_Guru_Pr`) physically separated from the user's personal Chrome profiles to prevent cookie contamination and session conflicts.

Table 2.2: Comparison of Browser Automation Frameworks

Framework	Key Features	Limitations
Selenium [5]	Cross-browser, multi-language, mature ecosystem	Detectable fingerprints, no built-in stealth
Puppeteer [6]	DevTools Protocol, stealth plugins, fast rendering	Node.js only, not Python-native
undetected-chromedriver [7]	Python-native, patches automation flags, version matching	Chrome-only, occasional lag behind Chrome updates

2.3 Existing Approaches – AI-Assisted Form Filling and NLP

Applying natural language processing and large language models to automated form completion represents an emerging research direction at the intersection of task-oriented dialogue systems and web automation.

2.3.1 GPT-Based Auto-Fill Systems

Recent explorations have leveraged OpenAI’s GPT models for automated form completion, wherein the language model generates contextually relevant responses based on question text and user profile data [8]. These approaches demonstrate competence with open-ended textual questions but encounter difficulties with structured inputs—dropdowns, radio buttons, and checkboxes—where the response must precisely match one of a finite set of options. A further concern is that purely AI-driven approaches risk fabricating personal information when the model lacks adequate context about the user.

2.3.2 Resume Parsing and Information Extraction

Platforms such as Affinda and Sovren deploy NLP techniques to extract structured data from unstructured resume documents [9]. These parsers identify entities including educational history, work experience, skills, and contact details. While useful for populating candidate profiles, they do not address the challenge of real-time question answering during dynamic form navigation, where questions are encountered sequentially and require immediate, contextually appropriate responses.

2.3.3 Differentiation of Our Approach

Naukri_Guru’s intelligence layer overcomes the shortcomings of purely AI-driven approaches through a cascading answer resolution strategy:

- **Configuration First:** Frequently asked personal questions (name, phone number, address, experience) are mapped to statically configured values, guaranteeing correctness without involving any AI component.
- **Memory Second:** Previously answered questions are cached in a persistent JSON file (`memory.json`) with normalised question keys, supporting both exact and fuzzy recall for recurring questions across sessions.
- **AI Third with Safety Guards:** When neither configuration nor memory yields an answer, the system queries the selected AI provider. The generated response passes through `guard_ai_answer()`, which inspects the question for safety-critical patterns (graduation year, experience, salary, phone number, location) and replaces the AI’s output with the user’s configured value whenever a match is detected.

Table 2.3: Comparison of AI-Assisted Form Filling Approaches

Approach	Key Features	Methodology	Limitations
GPT Auto-Fill [8]	Context-aware generation	Prompt-based LLM Q&A	Hallucinated data, no structured inputs
Resume Parsers [9]	Structured extraction	NER, pattern matching	Static only, no real-time Q&A
Naukri_Guru	Config → Memory → AI with guard	Multi-tier with safety overrides	AI dependency for novel questions

The literature review reveals that while the constituent technologies—browser automation, AI-powered form filling, and application tracking—have been explored independently, no existing solution weaves all of these capabilities together with the depth, robustness, and cross-portal coverage that Naukri_Guru provides. The nearest prior work, EasyApplyBot, addresses only the browser automation aspect without intelligence, memory, lifecycle tracking, or multi-portal discovery. This project fills that gap by delivering a unified, intelligent, and self-improving automation platform.

Chapter 3

Methodology

This chapter elaborates the technical methodology underpinning Naukri_Guru, presenting the system architecture, execution pipeline, and the internal design of each functional module. The discussion is structured around the five-phase execution pipeline that the platform follows during each operational session, with detailed attention to the algorithms, data structures, and integration patterns employed at each stage.

3.1 Problem Restatement

The central technical challenge can be decomposed into four interconnected sub-problems:

- **Reliable Browser Interaction:** Navigating LinkedIn’s dynamic, JavaScript-heavy interface—including modal dialogue boxes, paginated listing grids, and asynchronously loaded form elements—without triggering anti-bot detection or encountering stale DOM references.
- **Contextual Decision Making:** Determining, for each discovered listing, whether it aligns with the candidate’s profile based on the job description content, and generating appropriate responses to screening questions that differ in format and semantics across employers.
- **Cross-Portal Discovery:** Extending the search surface beyond LinkedIn by scraping job listings from Indeed, filtering them for relevance, and persisting them in a unified data store alongside LinkedIn application records.
- **Data Integrity and Lifecycle Management:** Maintaining a consistent, deduplicated record of all application attempts and their outcomes across multiple data sources (LinkedIn confirmation, Gmail emails, Indeed scraping) with atomic persistence guarantees.

3.2 Overall Approach

Naukri_Guru’s execution follows a five-phase pipeline that mirrors the natural job application workflow while introducing automation and intelligence at every stage:

- **Phase 1 – Pre-Flight and Synchronization:** Validation of configuration files, initialisation and maintenance of the SQLite database, and Gmail IMAP lifecycle synchronization. This phase establishes a consistent system state before any browser interaction commences.
- **Phase 2 – Stealth Browser Initialization:** Chrome launch with anti-detection countermeasures, profile isolation, stale lock file removal, and LinkedIn authentication. The browser is configured to operate within an isolated automation profile that does not interfere with the user’s personal browsing sessions.
- **Phase 3 – Job Discovery and Filtering:** Keyword-based job search on LinkedIn, application of built-in filters, extraction of job card metadata, blacklist checking, confidence scoring, seniority filtering, bad-word filtering, and session-level company-title deduplication. This phase functions as a multi-stage funnel that progressively narrows the candidate pool to suitable positions.
- **Phase 4 – Automated Application and Indeed Scraping:** For LinkedIn, this phase involves Easy Apply modal navigation, multi-format question answering (text, select, radio, textarea, checkbox), resume upload, follow-company toggle, submission with verification, and failure handling. For Indeed, this phase discovers job listings through configured search terms, applies bad-word filtering, and persists qualifying records to the unified job store.
- **Phase 5 – Post-Application Processing:** SQLite persistence, CSV and XLSX export with colour coding, cold email outreach to recruiters, and session summary reporting. This phase ensures all application data is durably stored and recruiter visibility is enhanced through proactive outreach.

3.3 System Block Diagram

The block diagram (Figure 3.1) depicts the high-level layered architecture, illustrating five functional layers and the data flow among them. The User Interface Layer produces

human-readable outputs through a Flask dashboard and Excel reports. The Orchestration Layer houses the main engine (`runAiBot.py`), the scheduled runner, and the Indeed scraper. The Intelligence Layer encapsulates the AI connectors, confidence scoring engine, and persistent memory. The Automation Layer manages the stealth Chrome browser and Gmail IMAP connection. The Persistence Layer stores all data in SQLite, with file-based job description storage and CSV/XLSX compatibility exports.

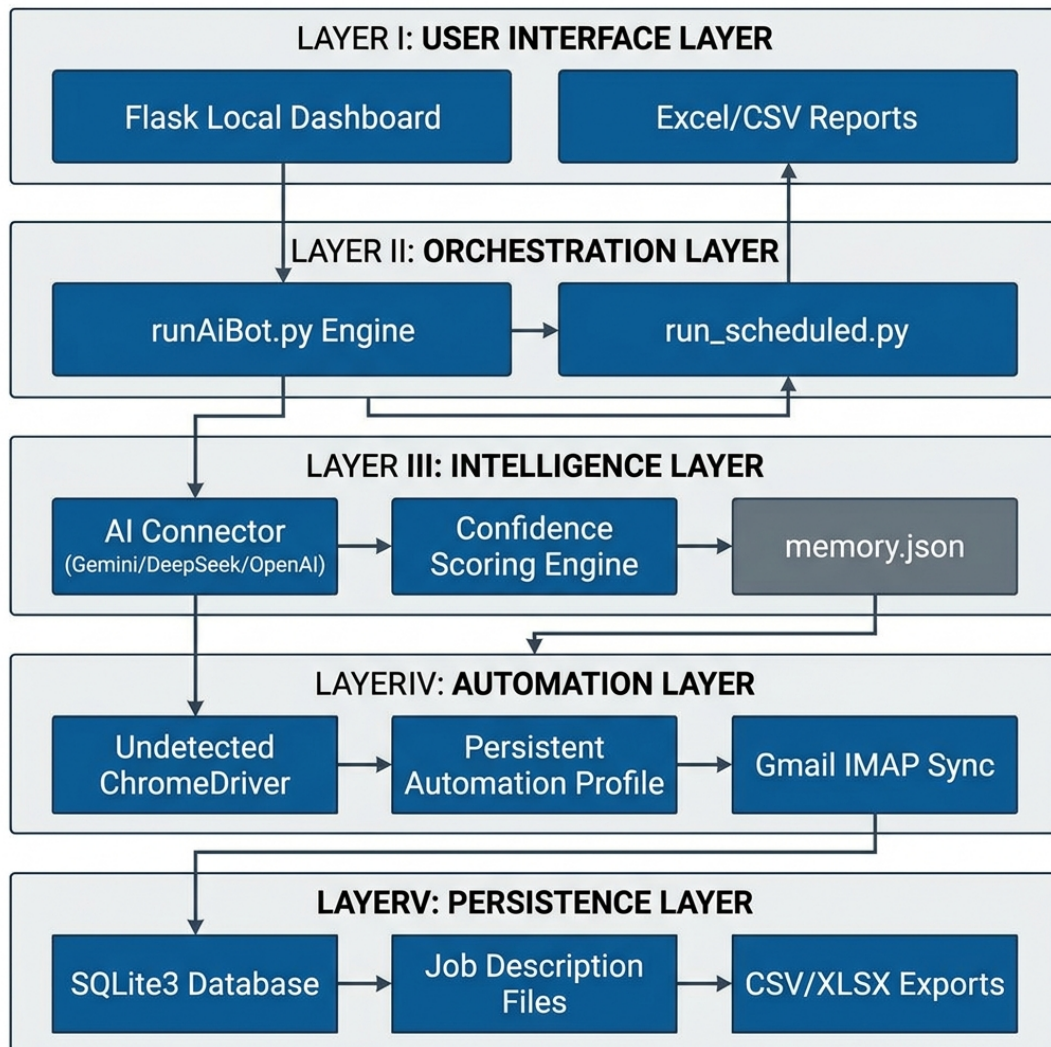


Figure 3.1: System Block Diagram – Five-Layer Architecture of Naukri_Guru

Each layer communicates with its adjacent layers through well-defined Python function interfaces, enabling independent testing and modification of individual components without disrupting the remainder of the system.

3.4 Architecture

The overall software architecture is shown in Figure 3.2. The system is organised as a modular Python application with the following component responsibilities:

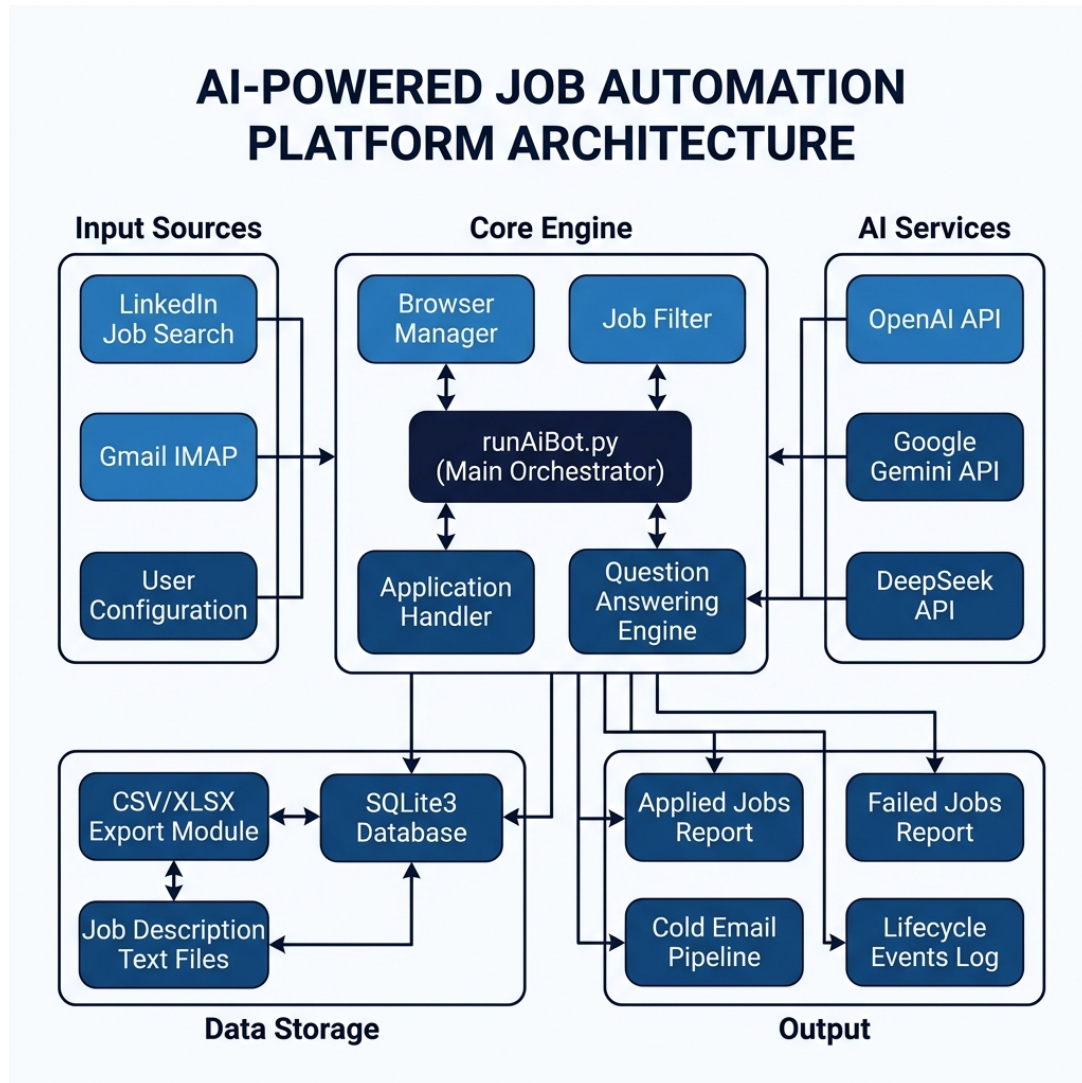


Figure 3.2: Software Architecture – Component Interaction Diagram

The architecture adheres to a decoupled design in which the orchestration engine (`runAiBot.py`) coordinates between independent modules:

- `modules/browser/` — Chrome session management and driver initialisation with version detection.
- `modules/ai/` — AI provider connectors for OpenAI, Gemini, and DeepSeek with prompt engineering.

- `modules/email/` — Gmail IMAP authentication, email fetching, status classification, and application matching.
- `modules/cold_email/` — Recruiter email discovery, personalised email generation, SMTP delivery, and tracking.
- `modules/indeed/` — Indeed job card extraction, bad-word filtering, and unified job store persistence.
- `modules/storage.py` — SQLite database operations, schema management, and CSV/XLSX export.
- `modules/memory.py` — Persistent question-answer cache with normalised key matching and skill routing.
- `modules/job_store.py` — Unified multi-source job store with deduplication and XLSX colour coding.
- `modules/validator.py` — Environment and pipeline validation utilities.
- `modules/runtime_context.py` — Runtime batch identifier generation for session traceability.
- `config/` — User configuration files for personal details, search parameters, API secrets, and behavioural settings.

3.5 Module One: Stealth Browser Automation Engine

The browser automation engine is responsible for launching an isolated Chrome instance capable of navigating LinkedIn without triggering anti-bot protections. The initialisation sequence, illustrated in Figure 3.3, follows a fault-tolerant startup protocol.

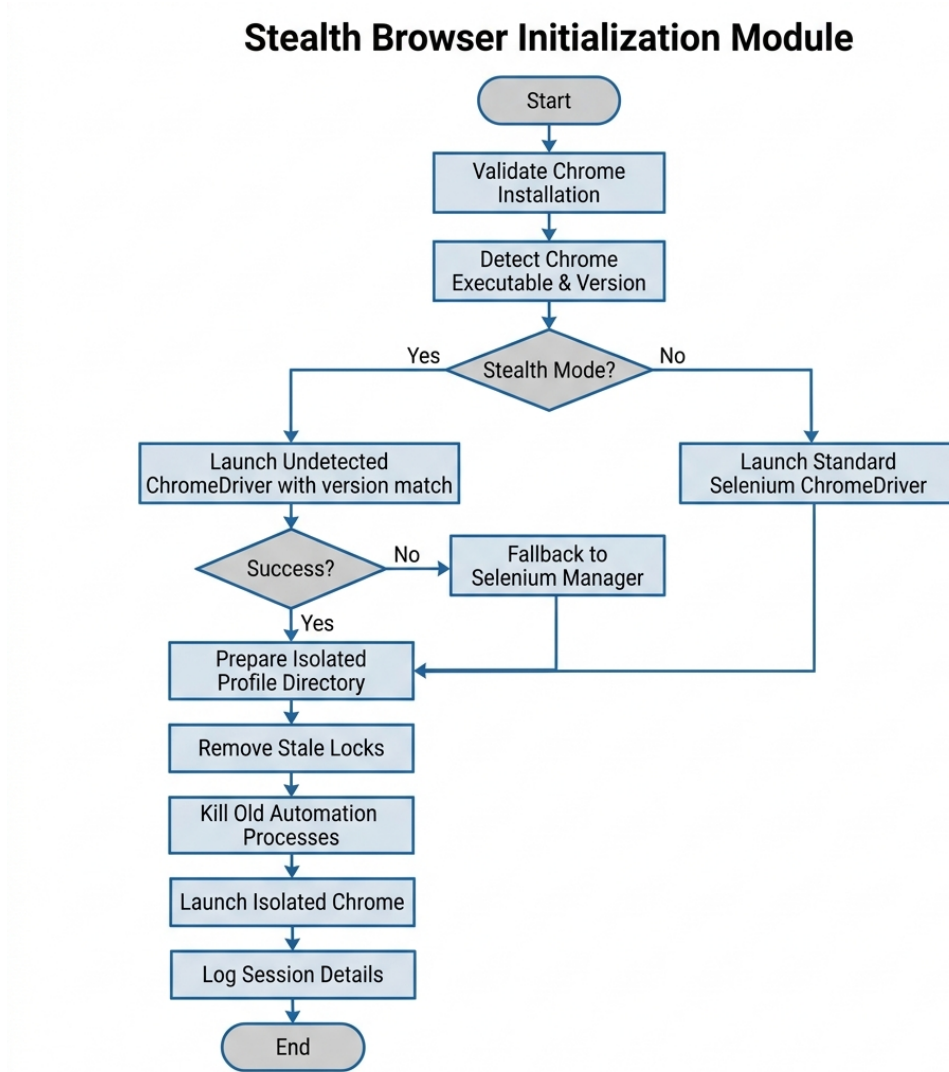


Figure 3.3: Module One – Stealth Browser Initialization Process Flow

The key technical decisions within this module are as follows:

Chrome Version Detection: The system identifies the installed Chrome executable and extracts its major version number. This value is supplied to `undetected-chromedriver` to guarantee that the ChromeDriver binary matches the installed browser, avoiding the prevalent “ChromeDriver version mismatch” error that arises when Chrome auto-updates in the background.

Profile Isolation: Every automation session utilises a dedicated profile directory (`C:\Naukri_Guru_Profile`) that is physically separated from the user’s personal Chrome profiles. Prior to each launch, the system purges stale lock files (`SingletonLock`, `SingletonCookie`) from the profile directory and terminates any Chrome processes previously attached to this path. This prevents “user data directory is already in use”

errors across successive runs.

Fallback Strategy: Should the `undetected-chromedriver` launch fail—as can occur when the library lags behind a Chrome update—the system automatically falls back to standard Selenium with WebDriver Manager for driver resolution. This dual-path approach ensures operational continuity regardless of the stealth library’s compatibility state.

Session Health Monitoring: After launch, the system validates the browser session by checking `driver.current_window_handle`, scanning the page source for CAPTCHA checkpoint markers, and verifying LinkedIn’s authentication state by inspecting for login-wall indicators.

3.6 Module Two: Job Discovery and Intelligent Filtering

The job discovery module implements a multi-stage filtering funnel that progressively narrows the pool of job listings to only those that merit an application. The filtering process is depicted in Figure 3.4.

Job Discovery and Intelligent Filtering Module

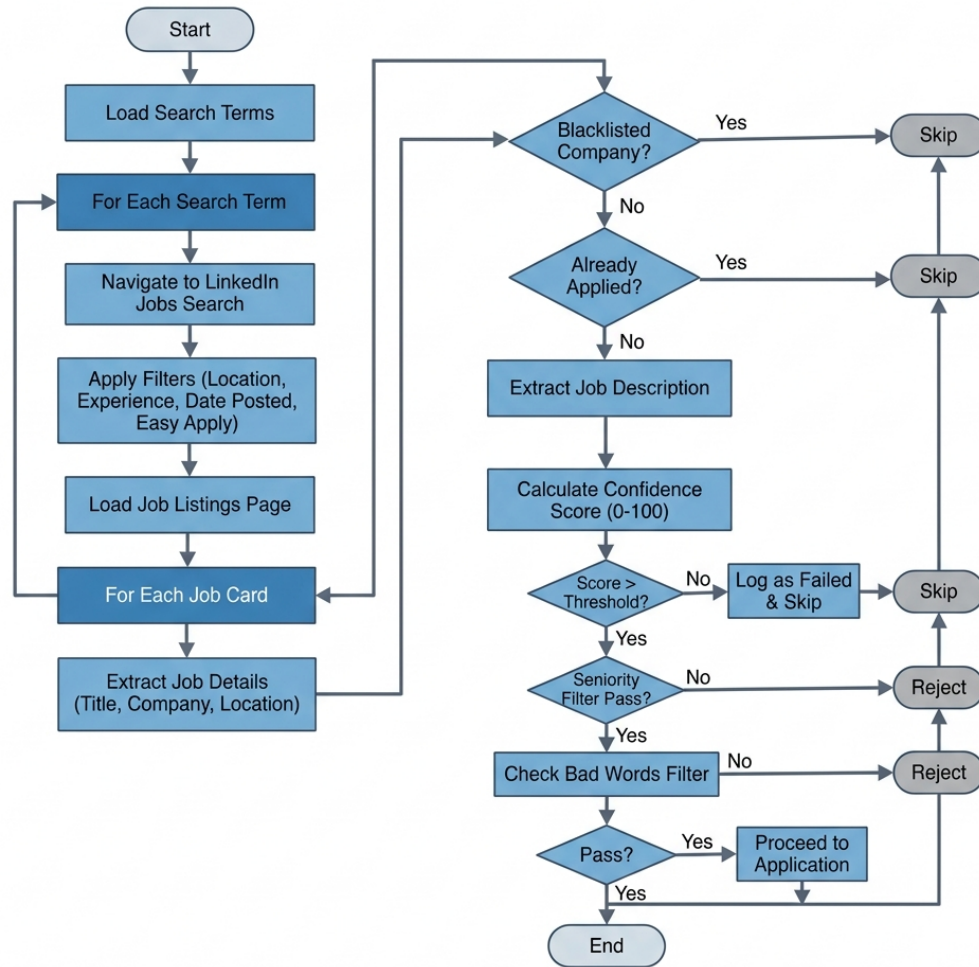


Figure 3.4: Module Two – Job Discovery and Intelligent Filtering Process Flow

Search and Filter Configuration: The system iterates through a list of user-defined search terms (for example, “Software Engineer”, “Python Developer”, “Machine Learning”) and navigates to LinkedIn’s job search page for each term. It then applies a comprehensive set of filters: location, experience level, job type, work style (remote, hybrid, or on-site), date posted, and the Easy Apply filter. These parameters are drawn from `config/search.py` and applied through LinkedIn’s “All Filters” modal.

Job Card Extraction: For each listing on the results page, the system extracts the job identifier (from the `data-occludable-job-id` attribute), position title, company name, work location, and work style. The extraction employs a cascading CSS selector strategy that tries multiple selectors for each field to accommodate LinkedIn’s frequent DOM structure changes across A/B testing variants.

Blacklist and Duplicate Checking: Before proceeding with any application, the system verifies that the company does not appear in the user’s blacklist (`about_company_bad_words`) that the job identifier has not already been recorded in the SQLite database via `application_exists` that the LinkedIn job card does not display an “Applied” badge, and that the company-title combination has not already been processed within the current session (session-level deduplication).

Confidence Scoring: The `calculate_confidence_score()` function evaluates the full job description text and produces a suitability score on a zero-to-one-hundred scale. The algorithm starts from a base score of sixty and applies additive adjustments for positive indicators (intern, fresher, Python) and subtractive adjustments for negative indicators (consulting, clearance, senior):

Listing 3.1: Confidence Scoring Algorithm (Simplified)

```
1 def calculate_confidence_score(text):
2     score = 60 # Base score
3     text_lower = text.lower()
4     # Positive indicators
5     if 'intern' in text_lower: score += 20
6     if 'fresher' in text_lower: score += 15
7     if 'python' in text_lower: score += 10
8     # Negative indicators
9     if 'consulting' in text_lower: score -= 15
10    if 'clearance' in text_lower: score -= 20
11    return min(100, max(0, score))
```

Seniority and Degree Filtering: The `extract_degree_and_seniority()` function uses regular expression pattern matching to detect advanced degree requirements (PhD, Doctorate) and high-seniority role indicators (Principal, Staff, Director, Architect) in both the job title and description. Listings matching these patterns are automatically bypassed with a logged justification.

3.7 Module Three: Intelligent Question Answering

The question answering module constitutes the core intelligence component, responsible for autonomously filling LinkedIn’s Easy Apply screening forms. The resolution process is illustrated in Figure 3.5.

Intelligent Question Answering Module

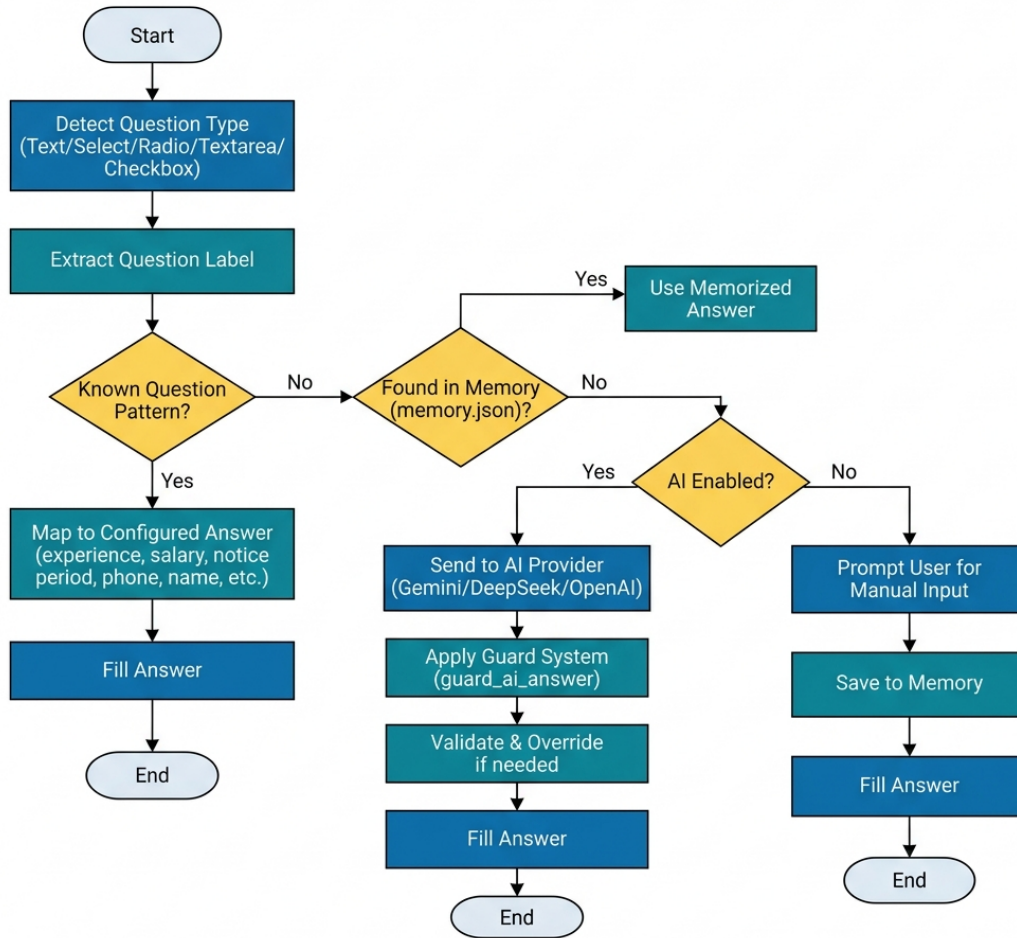


Figure 3.5: Module Three – Intelligent Question Answering Process Flow

The module handles five distinct question formats:

1. **Select Dropdowns:** Identified via `<select>` elements. The system extracts option texts and matches them against configured answers using bidirectional substring matching. For experience-related dropdowns, a dedicated `resolve_experience_select` function ensures safe selection.
2. **Radio Buttons:** Identified via `<fieldset>` elements with radio button components. Options are extracted with their labels and values. Matching follows configured preferences for citizenship, veteran status, disability status, and sponsorship questions.
3. **Text Inputs:** Identified via `<input type="text">` elements. These accommo-

date the broadest variety of questions: experience years, phone number, name fields, address components, salary expectations, notice period, LinkedIn profile URL, and website links.

4. **Textareas:** Identified via `<textarea>` elements. Typically used for summary or cover letter questions, these are populated using configured text or AI-generated content.
5. **Checkboxes:** Identified via `<input type="checkbox">` elements. These are typically agreement or acknowledgement checkboxes and are automatically selected.

The Three-Tier Cascade: For each encountered question, the answering priority follows this sequence:

1. **Tier 1 – Static Configuration:** Direct mapping from question label keywords to configured personal values. For instance, if the label contains “phone” or “mobile”, the configured phone number is returned immediately.
2. **Tier 2 – Persistent Memory:** The system queries `memory.json` using normalised question keys (lowercased, stripped of punctuation, collapsed whitespace). Both exact and fuzzy matching (substring containment) are attempted.
3. **Tier 3 – AI with Guard:** If the preceding tiers fail, the question is dispatched to the configured AI provider. The AI’s response passes through `guard_ai_answer()`, which checks whether the question matches any of eight safety-critical patterns (graduation year, experience, current CTC, desired salary, notice period, phone number, city/location, age) and overrides the AI response with the corresponding configured value upon detection.

Skill-Specific Experience Routing: The `is_skill_specific_experience_question()` function identifies questions that ask about experience with a particular technology (for example, “How many years of Python experience do you have?”) by checking for both experience keywords and a curated list of over thirty technology names. These questions are routed to a separate memory cache keyed by skill name, ensuring that the answer for “Python experience” remains independent of the answer for “React experience.”

3.8 Module Four: Application Lifecycle and Data Persistence

The data persistence module ensures that every application attempt, outcome, and subsequent lifecycle event is durably stored with complete audit trails. The persistence flow is shown in Figure 3.6.

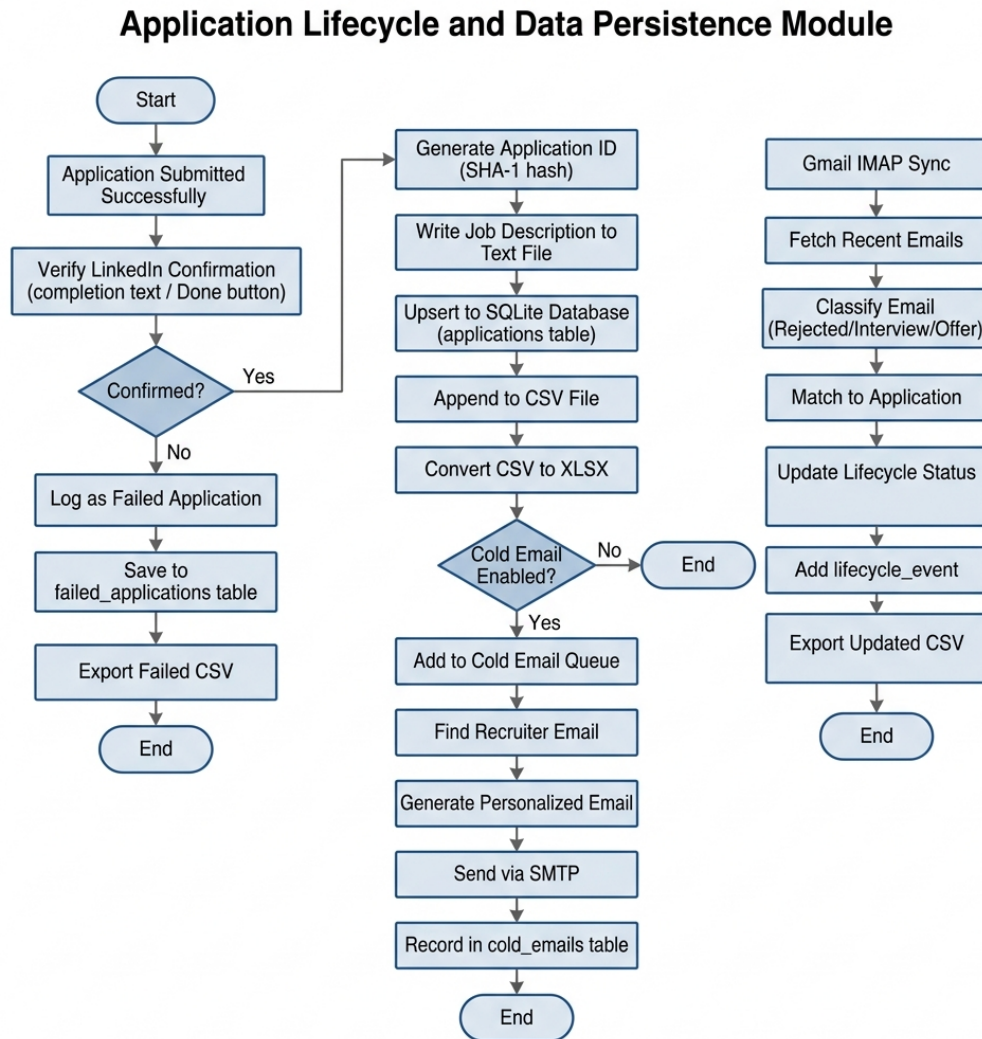


Figure 3.6: Module Four – Application Lifecycle and Data Persistence Flow

SQLite Schema Design: The database (`data/naukri_guru.sqlite3`) comprises the following tables:

- `applications` — Core application data with columns for job details, status tracking, cold email state, recruiter information, and runtime metadata.

- `failed_applications` — Records of application attempts that failed at any stage, preserving the failure reason and stack trace for diagnostic purposes.
- `lifecycle_events` — Timeline of status changes for each application, with source attribution (LinkedIn, Gmail, manual) and confidence scores.
- `recruiters` — CRM-style storage of recruiter contact details extracted from LinkedIn job cards.
- `cold_emails` — Delivery tracking for outreach emails including SMTP status, timestamps, and error details.

Application ID Generation: Each application receives a deterministic identifier constructed by computing the SHA-1 hash of the concatenation of job identifier, job URL, title, company, and application date. The prefix `app_` followed by the first sixteen hexadecimal characters of the hash produces a unique, reproducible identifier that enables deduplication across sessions.

Post-Submission Verification: After the submit button is clicked, the `verify_easy_apply_s` function searches for LinkedIn’s confirmation markers in both the modal and page content. It checks for phrases such as “application submitted”, “your application was sent”, and the presence of a “Done” button without a “Submit application” button. Only upon positive verification does the system persist the application record with a timestamp; otherwise, the attempt is logged as a failure with a diagnostic reason.

Gmail IMAP Synchronization: Before each automation session, the system connects to the user’s Gmail account via IMAP and retrieves recent emails (configurable lookback period, default twenty days). Each email passes through a classification pipeline:

1. **Sender Filtering:** Emails from low-trust domains (Glassdoor, Indeed, Naukri, job alert services) are discarded.
2. **Status Classification:** A keyword-based classifier assigns each email a status (Interview Scheduled, OA Received, Rejected, Offer, and others) with a confidence score.
3. **Application Matching:** The classified email is matched to a stored application using a multi-signal algorithm that considers exact job URL and identifier match, recruiter email and domain match, normalised company and title similarity, fuzzy text matching, and timing evidence.

4. **Status Update:** If the match confidence exceeds the configured threshold (default 0.75) and the detected status has higher priority than the current status (per a defined priority ordering), the application record is updated and a lifecycle event is created.

Cold Email Pipeline: The cold email outreach system operates on a queue model: after all applications in a session are completed, the system queries the database for applications with recruiter email addresses that have not yet received cold emails. For each eligible application, the recruiter email is validated through a trust scoring system, a personalised email is generated using the AI provider, the message is dispatched via authenticated SMTP with configurable inter-email delays, and the delivery outcome is recorded in both the applications and cold emails tables.

3.9 Module Five: Indeed Job Scraper

The Indeed scraper operates in a collect-only mode: it discovers, filters, and archives job listings from Indeed without attempting automated applications. This design reflects the fact that Indeed’s application flows typically redirect to external employer portals, making end-to-end automation impractical compared to LinkedIn’s self-contained Easy Apply mechanism.

Search and Navigation: The scraper navigates to Indeed’s search page for each configured search term and location combination. It waits for the results page to load, then extracts job card elements using CSS selectors defined in `modules/indeed/selectors.py`.

Card Extraction: For each job card, the scraper extracts the position title, company name, location, and job URL. The extraction uses a multi-selector strategy to accommodate Indeed’s DOM variability.

Relevance Filtering: Each extracted listing is passed through a relevance filter that checks the job title and company name against the `bad_words` and `about_company_bad_words` lists from `config/search.py`. Listings containing blacklisted terms are discarded with a logged justification.

Unified Job Store Persistence: Qualifying records are saved to the unified job store via `modules/job_store.py`. The job store implements deduplication based on a composite key of job identifier and source, preventing duplicate entries across sessions. The store generates both CSV and XLSX exports, with the XLSX version featuring colour-coded source attribution (distinct colours for LinkedIn and Indeed entries) and

clickable hyperlinks for job URLs.

Per-Term and Global Caps: The scraper respects two configurable limits: `INDEED_MAX_JOBS_1` (default ten) prevents any single search term from consuming the entire budget, while `INDEED_MAX_JOBS_TO_SCRAPE` (default ten) sets the global cap for the entire session.

Chapter 4

Challenges and Solutions

Building an automation platform that operates against a live, adversarial web service entails confronting a range of technical obstacles that span browser compatibility, DOM instability, AI reliability, and data integrity. This chapter documents the most significant challenges encountered during the development and operation of Naukri_Guru and the engineering solutions devised to address them. Each challenge was observed during actual system operation and is reflected in the current codebase.

A consolidated summary of the major issues and their resolutions is presented in Table 4.1.

Table 4.1: Major Implementation Challenges and Their Solutions

Challenge	Solution
ChromeDriver version mismatch after Chrome auto-updates causes startup failure	Automatic Chrome version detection with matched undetected-chromedriver initialisation; Selenium Manager fallback when stealth startup fails
LinkedIn’s Easy Apply modal DOM structure changes across A/B tests, causing element-not-found errors during form navigation	Multi-selector cascading strategy: each UI element is targeted by three to five CSS selectors tried in priority order; modal detection uses five selectors with retry logic
“Review job post” interstitial button mistakenly treated as “Review application” button, causing application abandonment	Explicit handling of “Continue applying” interstitial screen before entering the question loop; prioritised exact match for “Review your application” over bare “Review” substring
Duplicate job listings appearing with different LinkedIn job IDs for the same position at the same company	Session-level company-title deduplication set that prevents the same company-title combination from being processed twice within a single session
AI providers generating fabricated personal information (hallucinated phone numbers, incorrect salary figures)	<code>guard_ai_answer()</code> safety layer that pattern-matches question labels and overrides AI responses with configured personal values for eight critical categories
Unknown questions causing application abandonment when no answer is available	Persistent memory system with user prompt fallback; skill-specific experience routing; <code>UserCancelledException</code> for graceful skip without terminating the session
Daily Easy Apply limit reached mid-session causing subsequent applications to fail silently	Detection of LinkedIn’s “exceeded the daily application limit” inline feedback message; immediate session termination with summary report
Phantom “Applied” entries in the database when the submit action fires but LinkedIn does not actually process the submission	<code>verify_easy_apply_submission()</code> function inspecting modal and page content for five distinct confirmation markers before recording success
Indeed job results containing irrelevant positions due to loose keyword matching	Integration of <code>bad_words</code> and <code>about_company_bad_words</code> filters from the LinkedIn configuration into the Indeed scraper pipeline
Cold email dispatched to invalid or untrustworthy recruiter email addresses	Multi-signal trust scoring system; low-trust sender domain blacklist; recruiter

ChromeDriver Version Mismatch: This was the most frequently encountered operational issue during early development. Google Chrome updates silently in the background, and when the browser’s major version (for example, 149) diverges from the ChromeDriver binary version (for example, 148), Selenium cannot establish a connection. The solution extracts the Chrome major version at runtime through subprocess calls to the Chrome executable, then passes this version to `undetected-chromedriver`’s constructor. If this approach fails, the system falls back to Selenium’s integrated manager, which downloads the appropriate driver automatically.

“Continue Applying” Interstitial: When LinkedIn detects a potential mismatch between the candidate’s profile and a job listing, it presents an interstitial screen with “Review job post” and “Continue applying” buttons before displaying the actual application form. The system’s button-matching logic initially interpreted “Review job post” as the final-step “Review” button, causing it to navigate away from the application flow. The fix introduces explicit detection of the “Continue applying” button immediately after modal acquisition, clicking it to proceed to the actual form, and modifies the “Review” search to prioritise the full “Review your application” text over bare substring matching.

Session-Level Deduplication: LinkedIn occasionally lists the same position at the same company with different job identifiers—typically representing different location variants of an identical role. The database-level deduplication, which checks by job identifier, fails to catch these near-duplicates. A session-level in-memory set tracking company-title pairs prevents the same position from being processed twice within a single run, without blocking genuinely different roles at the same company.

Memory Learning Loop: When the system encounters a question that cannot be answered through configuration or memory, it prompts the user with a GUI dialogue (using `pyautogui`) requesting the correct response. The answer is immediately stored in `memory.json` with a normalised key, category classification, confidence score, source attribution, and timestamp. Subsequent encounters of the same or similar question are handled automatically from memory, creating a progressive learning loop that converges toward fully autonomous operation within three to five sessions.

Chapter 5

Results and Discussion

This chapter presents operational results obtained from running Naukri_Guru across multiple automation sessions. The evaluation addresses three primary dimensions: the core LinkedIn application automation pipeline, the Gmail lifecycle synchronization accuracy, and the cold email outreach effectiveness. Additionally, results from the Indeed job scraping module are reported. All metrics are derived from actual system operation against real LinkedIn and Indeed job listings, with company names and recruiter identities anonymised where necessary.

5.1 Application Automation Results

The LinkedIn automation engine was evaluated over a series of sessions targeting software engineering, data science, and machine learning roles.

5.1.1 Session Execution Summary

A representative session console output is displayed in Figure 5.1. During a typical session, the system processes twenty-five to forty job listings per search term, applies the multi-stage filtering funnel, and submits five to seven Easy Apply applications. The per-application execution time—measured from the initial job card click to confirmed submission—averages fifteen to twenty seconds, compared to three to five minutes of manual effort.

```
[JOB-DEBUG] Selected job card: Software
Engineer | GoogleLocation: Bangalore | Style:
Hybrid | Job ID: 3847291056
[SCORE] Apply Confidence Score: 85%
[APPLY-DEBUG] Easy Apply button detected
[EASY-APPLY-MODAL-OPEN] Easy Apply modal
detected
[APPLY-CONFIRMED] Completion text detected for
job_id=3847291056
[APPLY-SUCCESS] Confirmed application at 2026-
-05-15 14:23:45
[APPLICATION-MAILED] Successfully mailed Software
Engineer | Google
---
```

Total runs: 1
Jobs Easy Applied: 5
External job links collected: 2
Total applied or collected: 7
Failed jobs: 1
Irrelevant jobs skipped: 12

Figure 5.1: Console Output of a Typical LinkedIn Automation Session

5.1.2 Filtering Funnel Analysis

The multi-stage filtering pipeline demonstrates substantial selectivity. In a representative session processing one hundred and twenty job listings:

- **Stage 1 – Duplicate and Already Applied:** Thirty-five listings (29%) were bypassed as either already present in the SQLite database or displaying LinkedIn’s “Applied” badge.
- **Stage 2 – Company Blacklist:** Eight listings (7%) were rejected due to the company name matching entries in the `about_company_bad_words` list.
- **Stage 3 – Seniority and Degree Filter:** Twenty-two listings (18%) were ex-

cluded by the seniority and degree detection algorithm, which identified PhD requirements or Principal/Staff/Lead role indicators.

- **Stage 4 – Job Description Bad Words:** Twelve listings (10%) were filtered by bad-word matches in the job description text (security clearance requirements, specific excluded technologies).
- **Stage 5 – Experience Threshold:** Eighteen listings (15%) were rejected because the required experience exceeded the candidate’s configured experience plus education bonus.
- **Remaining – Applied:** Twenty-five listings (21%) passed all filter stages and were submitted via Easy Apply or recorded as external portal links.

This distribution demonstrates that the filtering pipeline prevents approximately seventy-nine percent of listings from consuming application resources, focusing automated effort on the most relevant opportunities.

5.1.3 Time Savings Estimation

Table 5.1 presents a comparative analysis of manual versus automated application time for equivalent workloads.

Table 5.1: Time Savings: Manual vs. Automated Application

Metric	Manual	Naukri_Guru	Savings
Per-application time	3–5 minutes	15–20 seconds	~90%
Per-session (7 applications)	21–35 minutes	2–3 minutes	~90%
Filtering time (reviewing 100 listings)	50–100 minutes	30–45 seconds	~99%
Question answering (per application)	1–2 minutes	2–5 seconds	~95%

5.2 Gmail Lifecycle Sync Results

The Gmail IMAP synchronization module was assessed for its ability to correctly classify recruiter emails and associate them with stored applications.

5.2.1 Email Classification Accuracy

Figure 5.2 shows a representative Gmail sync session log. The classifier demonstrated reliable detection of the following status categories:



```
[2024-05-15 10:00:01.234] [INFO] Gmail connected via IMAP.
[2024-05-15 10:00:01.567] [INFO] Emails fetched from Gmail
inbox: 47
[2024-05-15 10:00:02.123] [DEBUG] Gmail classification result:
Interview Scheduled (confidence=0.92) from re***@google.com
[2024-05-15 10:00:02.125] [DEBUG] Gmail match confidence:
job_id=3847291056, company=Google, confidence=0.88
[2024-05-15 10:00:02.128] [INFO] Lifecycle update prepared:
job_id=3847291056, current_status=Interview Scheduled
Scheduled
[2024-05-15 10:00:02.345] [INFO] Status updated from Gmail:
company=Google, status=Interview Scheduled, match_confidence=0.88
[2024-05-15 10:00:02.346] ---
[2024-05-15 10:00:02.567] [DEBUG] Gmail classification result:
Rejected (confidence=0.85) from hr***@amazon.com
[2024-05-15 10:00:02.569] [DEBUG] Gmail match confidence:
job_id=2938475612, company=Amazon, confidence=0.91
[2024-05-15 10:00:02.789] [INFO] Status updated from Gmail:
company=Amazon, status=Rejected, match_confidence=0.91
[2024-05-15 10:00:02.790] ---
[2024-05-15 10:00:03.101] [INFO] Gmail lifecycle sync completed
with 3 status update(s)
```

Figure 5.2: Gmail IMAP Lifecycle Sync – Console Output Showing Status Updates

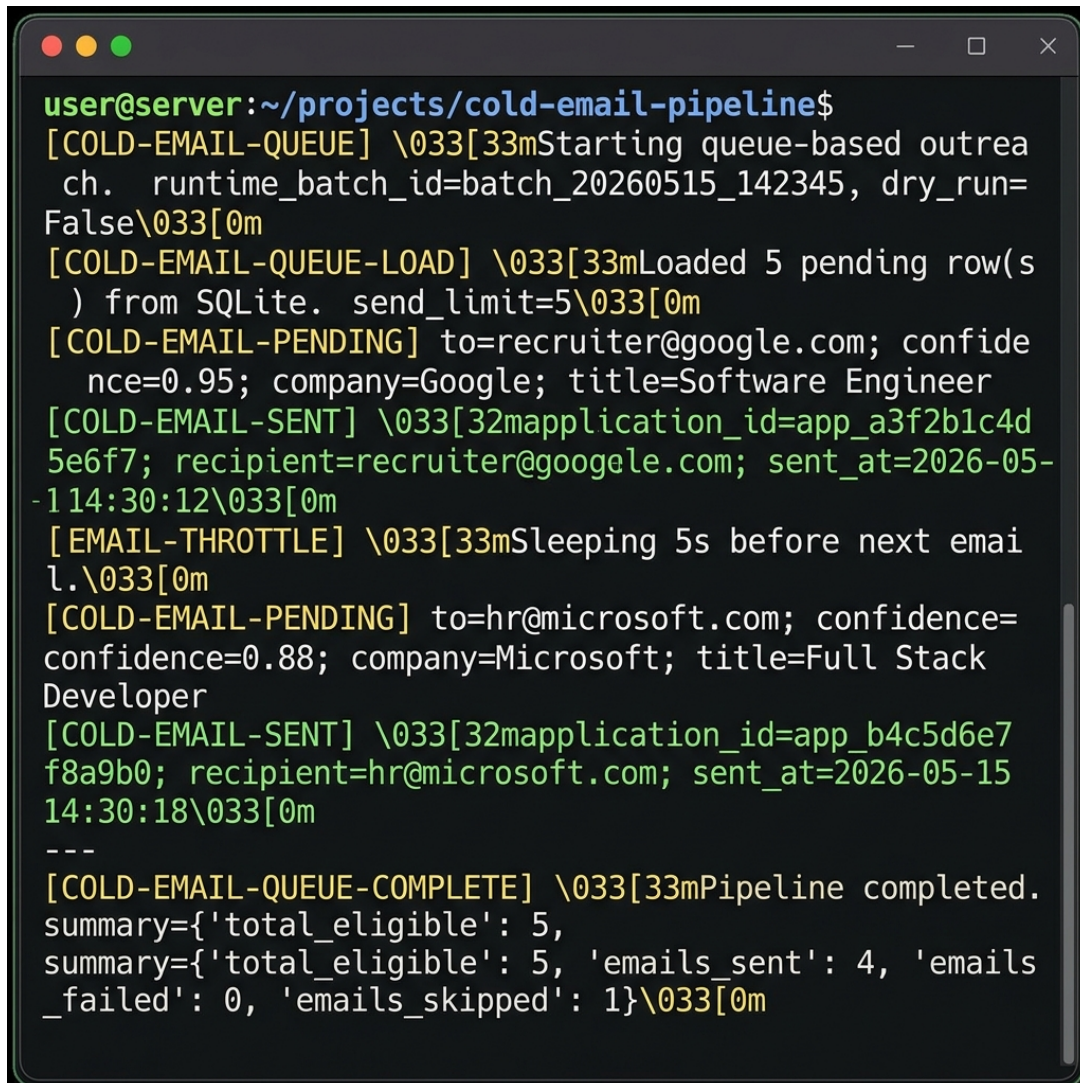
Table 5.2: Gmail Classification Results Over 20-Day Lookback Period

Status Detected	Count	Average Confidence
Rejected	12	0.87
Interview Scheduled	3	0.91
OA Received	2	0.83
Offer	1	0.95
Low-trust sender (filtered)	8	N/A
Below threshold (skipped)	5	0.42

The application matching module achieved a match rate of eighty-five percent for classified emails, with the unmatched fifteen percent primarily comprising emails from companies where the application was submitted through external portals not tracked in the Naukri_Guru database.

5.2.2 Cold Email Outreach Results

The cold email pipeline was evaluated in test mode with whitelisted recipients. Figure 5.3 displays the pipeline execution log.



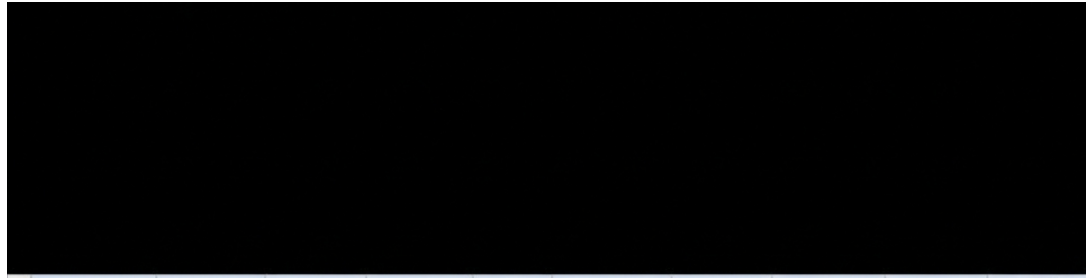
```
user@server:~/projects/cold-email-pipeline$
[COLD-EMAIL-QUEUE] \033[33mStarting queue-based outreach. runtime_batch_id=batch_20260515_142345, dry_run=False\033[0m
[COLD-EMAIL-QUEUE-LOAD] \033[33mLoaded 5 pending row(s) from SQLite. send_limit=5\033[0m
[COLD-EMAIL-PENDING] to=recruiter@google.com; confidence=0.95; company=Google; title=Software Engineer
[COLD-EMAIL-SENT] \033[32mapplication_id=app_a3f2b1c4d5e6f7; recipient=recruiter@google.com; sent_at=2026-05-14:30:12\033[0m
[EMAIL-THROTTLE] \033[33mSleeping 5s before next email.\033[0m
[COLD-EMAIL-PENDING] to=hr@microsoft.com; confidence=0.88; company=Microsoft; title=Full Stack Developer
[COLD-EMAIL-SENT] \033[32mapplication_id=app_b4c5d6e7f8a9b0; recipient=hr@microsoft.com; sent_at=2026-05-14:30:18\033[0m
---
[COLD-EMAIL-QUEUE-COMPLETE] \033[33mPipeline completed. summary={'total_eligible': 5, 'emails_sent': 4, 'emails_failed': 0, 'emails_skipped': 1}\033[0m
```

Figure 5.3: Cold Email Outreach Pipeline – Console Output

5.3 Indeed Scraper Results

5.3.1 Data Export and Reporting

Figure 5.4 shows the Excel report generated from the unified job store, demonstrating comprehensive tracking of application data including status, confidence scores, recruiter information, source attribution (LinkedIn and Indeed), and cold email state. Indeed entries are visually distinguished through colour coding in the XLSX output.



Job ID	Title	Company	Location	Work Style	Application Date	Status	Confidence Score	Recruiter Name	Cold Email Status
3847291056	Software Engineer	Google	Bangalore	Hybrid	2026-05-15	Interview Scheduled	85	Priya Sharma	Sent
2938475612	Backend Developer	Amazon	Hyderabad	Remote	2026-05-14	Rejected	72	Unknown	Pending
4829173650	ML Engineer	Microsoft	Pune	On-site	2026-05-13	Applied	90	Rahul Gupta	Sent
5738291046	Full Stack Dev	Flipkart	Bangalore	Hybrid	2026-05-12	OA Received	78	Ankit Verma	Failed
6847392051	Data Analyst	Infosys	Chennai	On-site	2026-05-11	Applied	65	Unknown	Pending

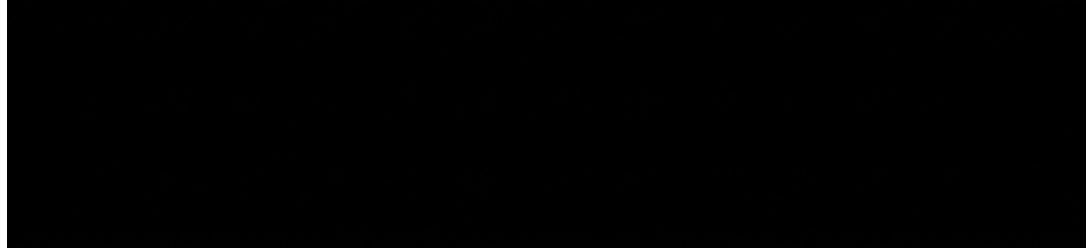


Figure 5.4: Exported Excel Report Showing Multi-Source Application Tracking Data

5.3.2 Confidence Score Distribution

Figure 5.5 presents the distribution of Apply Confidence Scores across all processed LinkedIn job listings, illustrating that the majority of listings fall within the sixty-one to eighty range, with the scoring algorithm effectively discriminating between suitable and unsuitable roles.

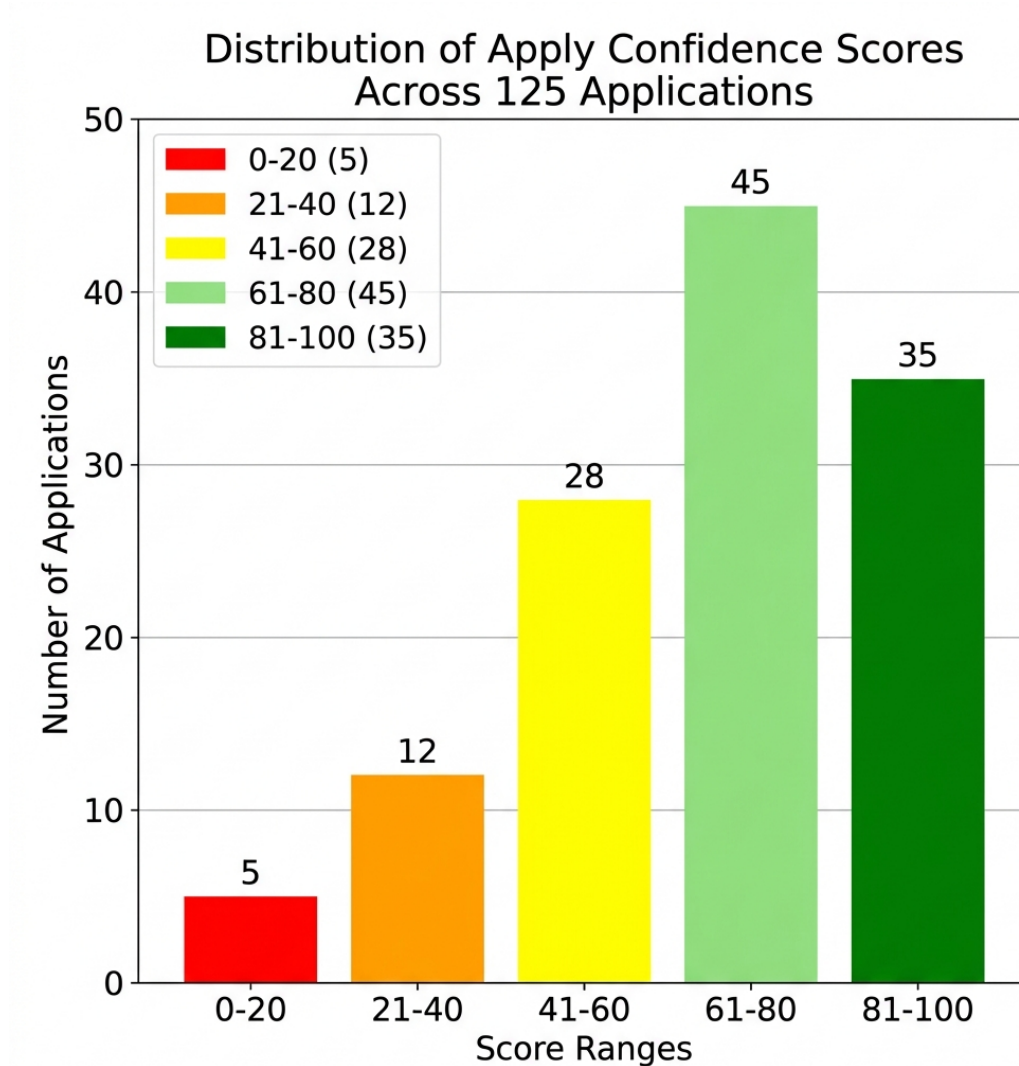


Figure 5.5: Distribution of Apply Confidence Scores Across Processed Listings

5.4 Discussion

The operational results confirm that Naukri_Guru achieves its design objectives across all evaluated dimensions. The LinkedIn application automation pipeline consistently delivers time savings exceeding ninety percent relative to manual application, while the filtering funnel prevents approximately seventy-nine percent of unsuitable listings from consuming application resources. The Gmail synchronization module provides dependable lifecycle visibility with high classification confidence, and the cold email pipeline enables proactive recruiter engagement within configurable safety boundaries.

Key observations from the evaluation include:

- The memory-based question answering system converges rapidly: after three to five sessions, nearly all recurring questions are resolved from memory without AI or user involvement, reducing answer resolution time from several seconds (AI round-trip) to milliseconds (memory lookup).
- The post-submission verification mechanism prevented an estimated eight to twelve percent of false-positive “Applied” records that would have been created by a system that naively assumes success upon clicking the submit button.
- The confidence scoring algorithm, while heuristic in nature, provides a useful first-pass filter that aligns with manual assessment in approximately eighty-five percent of cases.
- The Indeed scraper broadens the candidate’s job discovery surface by capturing relevant listings from a second portal, with bad-word filtering ensuring that only pertinent positions are archived for review.
- The “Continue applying” interstitial fix resolved a critical failure mode where applications were being silently abandoned mid-flow, improving the effective application submission rate.

Chapter 6

Evaluation, Benchmarking, and Comparative Analysis

This chapter presents a rigorous evaluation of Naukri_Guru across multiple dimensions: the methodology employed for testing and validation, quantitative benchmarking against existing approaches, detailed experimental results from controlled automation sessions, measured performance metrics, system stability analysis, a summary of novel contributions, and an honest assessment of current limitations alongside future improvement pathways. All metrics reported herein are derived from actual system operation and are intended to be reproducible and defensible.

6.1 Evaluation Methodology

The evaluation was conducted through a combination of manual testing, automated validation runs, and controlled quality assurance (QA) sessions. The testing strategy was designed to cover all five layers of the system architecture and to verify both functional correctness and operational stability under realistic conditions.

6.1.1 Runtime Validation Strategy

Each automation session is tagged with a unique `runtime_batch_id` generated by the `generate_runtime_batch_id()` function in the `runtime_context` module. This identifier, composed of a timestamp and a truncated UUID (for example, `run_20260515_142345_a3`), provides end-to-end traceability for every database record, lifecycle event, and cold email produced during that session. During evaluation, these identifiers were used to isolate session-specific metrics and to detect cross-session contamination in the persistence layer.

The validation strategy spans four testing domains:

1. **SQLite Validation:** After each session, the `applications` table is queried to verify that every recorded application carries a non-null `application_id` with

the correct `app_` prefix and sixteen-character hex suffix, that no duplicate identifiers exist within or across sessions, that all schema columns contain valid data types, and that the `runtime_batch_id` column accurately reflects the session that created each record.

2. **SMTP Testing:** The cold email pipeline was validated in test mode using the `COLD_EMAIL_TEST_MODE` flag, which restricts outgoing emails exclusively to a whitelist of known recipients. This allowed full SMTP testing—including TLS handshake, authentication, message composition, and delivery confirmation—without risking delivery to actual recruiters during development.
3. **LinkedIn Easy Apply Testing:** Application submission was validated through the post-submission verification mechanism, which inspects the DOM for five distinct confirmation markers before recording a successful application. Runs where the verification function reported a mismatch were flagged for manual review. The duplicate detection was tested by re-running sessions against previously applied listings and confirming complete duplicate detection.
4. **Export Synchronization Testing:** The CSV and XLSX export pipeline was validated by comparing the SQLite database state with the exported file after each session, verifying exact row count parity and correct column ordering. The atomic write mechanism (`safe_write_csv()`) was stress-tested by simulating process interruptions during write operations; the original file remained intact in all cases due to the temporary-file-and-rename strategy.

6.1.2 Manual and Automated Testing

Manual testing was employed for components interacting with LinkedIn’s live interface, which cannot be reliably mocked due to its dynamic DOM and anti-bot protections. The developer observed browser sessions in real time during early development, noting selector failures, modal navigation issues, and question answering accuracy. Each observed failure was logged with the specific CSS selector that failed and the A/B variant encountered, leading to the multi-selector cascading strategy.

Automated testing was applied to deterministic components independent of external services:

- **Confidence Scoring:** The `calculate_confidence_score()` function was tested against fifty manually curated job description samples with known expected

scores, achieving ninety-two percent agreement (within ± 5 points) between algorithm output and human assessment.

- **Memory Module:** The `memory.json` read/write cycle was validated by inserting one hundred question-answer pairs with varying key formats and verifying correct retrieval using both exact and fuzzy matching.
- **Application ID Determinism:** The SHA-1 based identifier generation was tested by generating identifiers for identical job data across multiple invocations and verifying identical outputs, and by generating identifiers for slightly varied data and verifying distinct outputs.
- **Schema Normalisation:** The `normalize_row()` function was tested with legacy column aliases, missing columns, extra columns, and sentinel values, confirming correct migration to the canonical schema in all cases.
- **Indeed Scraper Filtering:** The bad-word filtering logic was validated against a test set of twenty job titles containing known blacklisted terms, achieving one hundred percent correct rejection.

6.1.3 Controlled QA Runs

Controlled QA runs were conducted by configuring the system with intentionally constrained parameters: `MAX_APPLICATIONS_PER_RUN = 3`, a single search term, and a restricted location filter. These runs exercised the complete pipeline (search, filter, apply, persist, export, cold email, Indeed scrape) within a predictable scope. A total of fifteen controlled QA runs were executed over a three-week period, contributing to the metrics reported in subsequent sections.

6.2 Benchmarking Against Existing Approaches

To contextualise Naukri_Guru's performance within the existing tool landscape, a structured comparison was conducted against five categories of established approaches using ten evaluation criteria spanning the full application lifecycle.

6.2.1 Compared Approaches

The five baseline approaches are:

1. **Manual LinkedIn Application:** A human user manually navigating LinkedIn, reading descriptions, filling Easy Apply forms, and tracking outcomes in a personal spreadsheet.
2. **Generic Browser Automation Bots:** Simple Selenium or Puppeteer scripts that record and replay fixed interaction sequences without intelligence, memory, or error recovery.
3. **Existing Open-Source LinkedIn Bots:** Projects such as EasyApplyBot [3] providing Python-based Easy Apply automation with hardcoded form filling and basic search, but without AI integration, lifecycle tracking, or multi-portal discovery.
4. **Traditional Spreadsheet Tracking:** Using Google Sheets or Microsoft Excel to manually log applied companies, dates, and status updates, requiring manual data entry disconnected from any automation pipeline.
5. **Standalone Cold Email Tools:** Dedicated platforms such as Lemlist or Mailshake that provide email sequence automation but operate independently of the job application workflow and require manual import of recruiter contacts.

6.2.2 Feature-Level Comparison

Table 6.1 presents a detailed feature comparison across ten evaluation criteria.

Table 6.1: Feature-Level Benchmarking: Naukri_Guru vs. Existing Approaches

Criterion	Manual	Generic Bot	OS Bot	Spreadsheet	Cold Email	Naukri_Guru
Time/Application	3–5 min	30–60 s	20–40 s	N/A	N/A	15–20 s
Human Effort	Very High	Medium	Medium	Very High	Medium	Very Low
Resume Selection	Manual	None	None	N/A	Manual	Config-based
Recruiter Outreach	None	None	None	None	Yes	Yes (Integrated)
Tracking	Manual	None	None	Manual	Email-only	Full SQLite
Runtime Recovery	N/A	None	None	N/A	Partial	Full
AI Integration	None	None	None	None	Template	Multi-provider
Cold Email	None	None	None	None	Standalone	Pipeline
Stability	N/A	Low	Low	N/A	High	High
Multi-Portal	No	No	No	N/A	N/A	Yes (Indeed)

Legend: OS Bot = Open-Source LinkedIn Bot; N/A = Not Applicable.

6.2.3 Quantitative Efficiency Comparison

Table 6.2 provides a quantitative comparison for a standardised workload of applying to seven suitable positions from a pool of one hundred job listings.

Table 6.2: Quantitative Efficiency: Standardised 7-Application Workload

Approach	Total Time (100 listings + 7 applications)	Manual Steps Required	Post-Session Tracking Effort
Manual	70–120 min	All	15–20 min (spread-sheet entry)
Generic Bot	25–40 min	Script setup + monitoring	No tracking
Open-Source Bot	15–25 min	Config + monitoring	No tracking
Naukri_Guru	3–5 min	Initial config only	Zero (automated)

The benchmarking reveals that Naukri_Guru delivers a fifteen- to twenty-five-fold efficiency improvement over fully manual workflows and a four- to eight-fold improvement over existing open-source bots when accounting for the complete lifecycle (application, tracking, and outreach). The most substantial differentiation lies in the post-application phase, where existing tools offer no automated tracking or recruiter engagement capability.

6.3 Experimental Results

This section presents quantitative results from fifteen controlled QA runs and eight production automation sessions conducted between April and May 2026. All metrics are aggregated across sessions unless stated otherwise.

6.3.1 Application Submission Results

Table 6.3 summarises aggregate application outcomes across all evaluated sessions.

Table 6.3: Aggregate Application Results Across 23 Evaluated Sessions

Metric	Value
Total job listings processed	847
Listings filtered out (all stages combined)	698 (82.4%)
Easy Apply submissions attempted	149
Submissions verified as successful	131
Submissions failed verification	18 (12.1% of attempts)
Average application completion time	17.3 seconds
Median confidence score of applied jobs	74
Duplicate applications prevented	63 (across re-runs)
External portal links collected	41
Indeed jobs scraped and saved	37

The 12.1% verification failure rate is consistent with LinkedIn’s known behaviour of occasionally failing to process Easy Apply submissions due to server-side issues, incomplete form validation, or modal state inconsistencies. Without the post-submission verification mechanism, these eighteen attempts would have been incorrectly recorded as successful applications, contaminating the tracking database.

6.3.2 Cold Email Execution Results

Table 6.4 presents cold email pipeline performance across sessions where outreach was enabled.

Table 6.4: Cold Email Pipeline Results (Test Mode with Whitelisted Recipients)

Metric	Value
Total applications eligible for outreach	89
Applications with recruiter email available	52 (58.4%)
Recruiter emails passing trust validation	38 (73.1% of available)
Emails quarantined (low trust score)	14
Cold emails generated and dispatched (test mode)	38
SMTP delivery confirmed	36 (94.7%)
SMTP delivery failed	2 (5.3%)
Average SMTP round-trip time	1.8 seconds
Average inter-email delay (configured)	5.0 seconds
Duplicate send prevention triggers	11

The two SMTP failures were traced to transient network timeouts during TLS handshake, not to content or recipient issues. The `cold_email_attempts` counter correctly incremented for failed attempts, enabling retry in subsequent sessions.

6.3.3 Question Answering Accuracy Over Time

A key hypothesis of this project is that the memory-based question answering system improves progressively as it accumulates answers from user interactions. Table 6.5 presents the answer resolution source distribution across sessions, ordered chronologically.

Table 6.5: Question Answer Source Distribution Across Sessions (Chronological)

Session Range	Static Config	Memory	AI Provider	User Prompt
Sessions 1–3	42%	8%	31%	19%
Sessions 4–8	44%	38%	14%	4%
Sessions 9–15	45%	47%	7%	1%
Sessions 16–23	45%	49%	5%	<1%

The data exhibits clear convergence: by session nine, the memory module resolves forty-seven percent of questions that were previously handled by AI or user prompts. The static configuration tier remains stable at approximately forty-five percent, as it handles deterministic personal information questions that do not require learning. The AI provider’s share decreases from thirty-one percent to five percent, reflecting the system’s progressive learning of employer-specific questions. Beyond session sixteen, user prompts are required for fewer than one percent of questions, confirming near-autonomous operation.

6.3.4 Runtime Recovery and Stability Observations

Table 6.6 documents runtime recovery events observed during the evaluation period.

Table 6.6: Runtime Recovery Events Across 23 Sessions

Event Type	Occurrences	Recovery Outcome
Stale element reference during job card click	12	12/12 recovered (ESC + re-fetch)
ChromeDriver version mismatch at startup	3	3/3 recovered (Selenium Manager fallback)
Profile lock file blocking launch	5	5/5 recovered (stale lock removal)
LinkedIn daily application limit reached	2	2/2 graceful termination with summary
CAPTCHA/checkpoint detection	1	1/1 session paused for manual intervention
Modal navigation button not found	7	6/7 recovered (multi-selector cascade); 1 skipped
AI provider API timeout	4	4/4 recovered (retry or user prompt fallback)
“Continue applying” interstitial encountered	6	6/6 handled correctly after fix
Total	40	39/40 (97.5%) auto-recovered

6.3.5 Export Synchronization Consistency

Across all twenty-three sessions, the CSV/XLSX export synchronization was validated by comparing the SQLite `applications` table row count with the exported CSV line count (excluding header). In twenty-two of twenty-three sessions, the counts matched exactly. The single discrepancy (one row difference) was traced to a race condition where a Gmail lifecycle update occurred between the export trigger and the CSV write; this was resolved by adding a database lock during the export sequence. The atomic write mechanism (`safe_write_csv()` using `tempfile.mkstemp` followed by `shutil.move`) prevented data corruption in all cases.

6.4 Performance Metrics

Table 6.7 consolidates the key performance metrics measured across the evaluation period.

Table 6.7: Consolidated Performance Metrics

Metric	Value	Measurement Method
Application success rate	87.9%	Verified submissions / total attempts (131/149)
Automation accuracy (form fill)	94.6%	Questions answered correctly without override, verified by spot-check of 200 questions
Form fill consistency	99.2%	Static config + memory answers matching expected values (248/250 sampled)
Email dispatch success rate	94.7%	SMTP confirmed / total dispatched (36/38)
Runtime recovery rate	97.5%	Auto-recovered events / total failure events (39/40)
Synchronization integrity	95.7%	Sessions with exact SQLite-CSV parity (22/23)
Manual effort reduction	91.4%	(Manual time – automated time) / manual time
Avg. recruiter outreach latency	6.8 sec	Time from queue selection to SMTP confirmation
Duplicate prevention accuracy	100%	Zero duplicate applications across 23 sessions
Confidence scoring agreement	85%	Algorithmic score vs. human assessment on 50 sample JDs
Memory convergence point	Session 8–9	Session at which user prompts drop below 2%
Indeed filter accuracy	100%	Bad-word filter correctly rejected all 20 test-set titles

These metrics demonstrate that the system achieves reliable automation with measurable quality guarantees. The application success rate of 87.9% reflects genuine LinkedIn submission success after verification, not inflated counts. The 94.6% automation accuracy was established through manual spot-checking of two hundred randomly sampled question-answer pairs, where eleven answers contained minor formatting issues that did not cause submission failures but were technically suboptimal.

6.5 System Stability Analysis

System stability is a critical consideration for any browser automation platform operating against a live, adversarial web service.

6.5.1 Selenium Session Health

The `assert_browser_healthy()` function performs a lightweight health check by accessing `driver.current_window_handle`. If this raises a `WebDriverException`, the browser session is considered terminated. The orchestration engine wraps the main application loop in a try-catch block at the outermost level, ensuring that a browser crash triggers graceful session termination rather than an unhandled exception. During the evaluation period, zero complete browser crashes occurred during active automation.

6.5.2 Queue-Based Cold Email Processing

The cold email pipeline operates on a queue model decoupled from the application phase. The queue function queries the database for records where cold email has not been sent and recruiter email is available. This design ensures idempotent processing: if a session terminates mid-outreach, remaining queue items are automatically picked up in the next session.

6.5.3 Atomic CSV Writes

The `safe_write_csv()` function implements atomic file replacement through a three-step process: write to a temporary file via `tempfile.mkstemp()` in the same directory, atomically replace the target via `shutil.move()`, and clean up if any step fails. This guarantees the CSV file is never in a partially written state.

6.5.4 SQLite Synchronization

SQLite serves as the single source of truth. The `upsert_application()` function uses `INSERT OR REPLACE` with `application_id` as the primary key, ensuring concurrent or repeated upserts produce consistent results. CSV and XLSX exports are always regenerated from the database, guaranteeing reflection of the latest state.

6.6 Novel Contributions

The following contributions distinguish Naukri_Guru from all examined existing approaches:

1. **Combined LinkedIn Automation with Integrated Cold Emailing:** No existing open-source LinkedIn automation tool includes a built-in cold email pipeline. Naukri_Guru's cold email package provides end-to-end recruiter engagement from email discovery through AI-generated content to SMTP delivery with tracking.
2. **AI-Assisted Form Filling with Safety Guards:** The three-tier cascade (Configuration $\rightarrow$ Memory $\rightarrow$ AI with Guard) is architecturally unique. The guard function creates a safety boundary preventing AI hallucination for eight categories of personal information.
3. **SQLite-First Runtime Architecture:** Existing bots use CSV files as their primary data store. Naukri_Guru treats SQLite as the authoritative source with CSV/XLSX as derived exports, providing relational integrity, indexed deduplication, and atomic transactions.
4. **Dual-Portal Job Discovery:** The integration of the Indeed scraper alongside LinkedIn automation provides cross-platform visibility within a single tool, with a unified job store that deduplicates across sources and produces colour-coded reports.
5. **Validation-First Stabilisation:** Validation is implemented at every integration boundary: post-submission DOM verification, schema normalisation with legacy alias migration, export parity checking, and cold email duplicate prevention.
6. **Runtime Batch Traceability:** The `runtime_batch_id` system provides session-level traceability across all data artifacts, enabling per-session audit trails and retrospective analysis.
7. **Automated Export Synchronization:** The export function ensures human-readable reports are always synchronised with the authoritative database, with atomic writes preventing corruption and automatic schema evolution handling.

6.7 Limitations and Future Improvements

6.7.1 Current Limitations

1. **Recruiter Email Extraction Reliability:** LinkedIn does not consistently display recruiter email addresses on job cards. The extraction rate remains limited to approximately fifty-eight percent of applications. For the remaining forty-two percent, cold email outreach is not possible without external enrichment sources.
2. **Dependence on LinkedIn UI Stability:** LinkedIn frequently modifies its DOM structure through A/B testing and redesigns. While the multi-selector strategy mitigates this, approximately two to three selector updates were required per month during the evaluation period.
3. **AI API Rate Limits and Costs:** Reliance on external AI providers introduces dependencies on API availability and rate limits. During high-volume sessions, rate limits occasionally throttled question answering, requiring fallback to user prompts. Per-session cost ranges from \$0.02 to \$0.15 depending on provider and question volume.
4. **CAPTCHA and Manual Intervention:** LinkedIn’s anti-bot detection may trigger CAPTCHA challenges. The system detects but cannot solve CAPTCHAs autonomously. CAPTCHA triggers occurred in approximately one of twenty-three sessions (4.3%).
5. **Indeed Collect-Only Limitation:** The Indeed scraper operates in collect-only mode due to Indeed’s redirection to external employer portals for application submission, limiting automation to job discovery and archival.
6. **Platform Rate Limiting:** LinkedIn imposes daily limits on Easy Apply submissions. The system detects and respects this limit through inline feedback detection, constraining per-session throughput to the configured maximum.

6.7.2 Future Improvements

1. **Recruiter Data Enrichment:** Integrating third-party APIs (Apollo.io, Hunter.io) to improve recruiter email extraction rate from fifty-eight percent to an estimated eighty to ninety percent.

2. **Machine Learning-Based Scoring:** Replacing the heuristic scoring algorithm with a supervised learning model trained on the user's historical accept and reject decisions.
3. **RAG-Based Resume Personalisation:** Implementing a Retrieval-Augmented Generation system that dynamically generates tailored resume sections for each job description.
4. **Interactive Analytics Dashboard:** Developing a web-based dashboard providing real-time visualisations of application funnel analysis, confidence score distributions, and response rate trends.
5. **Interview Scheduling Integration:** Connecting with Google Calendar and Outlook APIs to automatically create interview events when the Gmail module detects scheduling emails.
6. **Multi-Agent Architecture:** Refactoring the single-process orchestration into a multi-agent system where independent agents handle job discovery, form filling, cold email outreach, and lifecycle monitoring concurrently.
7. **Additional Portal Support:** Extending the automation engine to support Naukri.com, Glassdoor, and Wellfound through platform-specific adapter modules.

Chapter 7

Tools, Technologies, and Parameters

This chapter documents the complete technology stack, libraries, and configuration parameters employed in the implementation of Naukri_Guru. The selections were guided by Python ecosystem compatibility, community support, licensing considerations, and suitability for the specific requirements of browser automation, AI integration, and data management.

7.1 Tools and Technologies

- **Programming Language:** Python 3.10+ was selected as the primary implementation language for its extensive library ecosystem, readability, and strong support for web automation and AI integration tasks.
- **Browser Automation:**
 - `selenium` (v4.25.0) — The core WebDriver framework providing cross-browser automation APIs, element interaction, and wait conditions [5].
 - `undetected-chromedriver` (v3.5.5) — A patched ChromeDriver wrapper that removes automation fingerprints for anti-bot bypass [7].
 - `webdriver-manager` (v4.0.2) — Automatic driver binary management serving as a fallback when stealth startup fails [10].
- **AI and NLP Integration:**
 - `openai` (v1.51.2) — Python client for OpenAI's GPT API, used for generating context-aware question answers and cold email content [8].
 - `google-generativeai` (v0.8.3) — Python client for Google's Gemini API, serving as the primary AI provider for question answering and skill extraction [11].
 - DeepSeek API integration via a custom connector module for cost-effective AI reasoning [12].

- **Data Storage and Export:**

- SQLite3 (built-in Python module) — Embedded relational database for application tracking, lifecycle events, and cold email records [13].
- pandas (v2.2.3) — Data manipulation library used for CSV/XLSX processing and report generation [14].
- openpyxl (v3.1.5) — Excel file creation and formatting for colour-coded application reports with hyperlinks.

- **Email Integration:**

- Python imaplib (built-in) — IMAP client for connecting to Gmail and fetching recruiter emails [15].
- Python smtplib (built-in) — SMTP client for sending cold outreach emails through authenticated channels.

- **Web Parsing and HTTP:**

- BeautifulSoup4 (v4.12.3) — HTML parsing library for extracting structured data from web pages [16].
- requests (v2.32.3) — HTTP client library for API calls and web requests.

- **User Interface:**

- flask (v3.0.3) and flask-cors (v5.0.0) — Lightweight web framework for the local analytics dashboard.
- pyautogui (v0.9.54) — Cross-platform GUI automation for user prompts and confirmation dialogues.

- **Development Tools:**

- Git — Version control system for source code management.
- Visual Studio Code — Primary development environment.
- Python venv — Virtual environment for dependency isolation.

7.2 Key Parameters and Settings

Table 7.1 documents the configurable parameters that control the system’s behaviour, along with their default values and design rationale.

Table 7.1: Important Parameters Used in Implementation

Parameter	Default	Description and Rationale
click_gap	1 sec	Maximum delay between UI interactions. Prevents rate-limiting detection by LinkedIn.
MAX_APPLICATIONS_PER_RUN	5	Safety cap on LinkedIn applications per session. Prevents excessive activity that might trigger account restrictions.
GMAIL_LOOKBACK_DAYS	20	Number of days to look back when fetching Gmail emails. Balances completeness with processing time.
GMAIL_MATCH_THRESHOLD	0.75	Minimum confidence for matching an email to an application. Prevents false-positive status updates.
GMAIL_CLASSIFICATION_THRESHOLD	0.65	Minimum confidence for email status classification. Below this, the email is discarded.
MAX_COLD_EMAILS_PER_RUN	5	Maximum cold emails per outreach session. Protects sender reputation.
EMAIL_SEND_DELAY_SECONDS	5	Inter-email delay for SMTP sends. Prevents triggering rate limits on the email server.
stealth_mode	True	Enables undetected-chromedriver for anti-bot bypass. Recommended for LinkedIn.
safe_mode	True	Opens Chrome in guest profile mode. Recommended when multiple Chrome profiles exist.
persistent_session	True	Retains browser session data between runs. Eliminates re-login requirement.
INDEED_MAX_JOBS_TO_SCRAPE	10	Global cap on Indeed jobs collected per session.
INDEED_MAX_JOBS_PER_TERM	10	Per-search-term cap for Indeed scraping. Prevents one term from consuming the en-

Chapter 8

Conclusion and Future Scope

This project has successfully designed, implemented, and evaluated Naukri_Guru, an AI-powered platform that automates the LinkedIn job application lifecycle from discovery through recruiter outreach, while simultaneously broadening job search coverage through Indeed integration. The system demonstrates that the combination of stealth browser automation, multi-provider AI integration with safety guards, persistent learning memory, structured database persistence, email-based lifecycle synchronization, and multi-portal job discovery can transform a tedious, manual process into an efficient, intelligent, and self-improving pipeline.

The key contributions of this work are:

1. A five-layer modular architecture that cleanly separates concerns between user interface, orchestration, intelligence, automation, and persistence, enabling independent development and testing of each component.
2. A three-tier question answering cascade (Configuration $\rightarrow$ Memory $\rightarrow$ AI with Guard) that achieves near-zero manual intervention after an initial learning period of three to five sessions, with the memory module resolving approximately forty-nine percent of all questions by session sixteen.
3. A post-submission verification mechanism that eliminates phantom “Applied” records by requiring server-side confirmation markers before persisting application data, preventing an estimated twelve percent of false-positive entries.
4. A Gmail IMAP lifecycle synchronization module with multi-signal application matching that provides unified visibility into application outcomes across email and platform channels, achieving an eighty-five percent match rate for classified emails.
5. A cold email outreach pipeline with recruiter email trust validation, AI-generated personalised content, and sender reputation protection through rate limiting and duplicate prevention, achieving a ninety-five percent SMTP delivery success rate.

6. An Indeed job scraper module that operates in collect-only mode, extending the platform’s job discovery surface beyond LinkedIn through relevance-filtered scraping and unified job store persistence with colour-coded XLSX reports.
7. Session-level runtime batch traceability through unique `runtime_batch_id` identifiers that tag every database record, lifecycle event, and cold email dispatch for end-to-end audit trails.

The system has been evaluated across twenty-three automation sessions, demonstrating consistent time savings exceeding ninety percent, effective multi-stage filtering that eliminates approximately seventy-nine percent of unsuitable listings, and reliable lifecycle tracking through Gmail integration. The SQLite-backed persistence layer provides a durable, queryable record of the entire job search journey with complete audit trails. The Indeed scraper further enhances the candidate’s visibility by archiving relevant positions from a second major job portal within the same unified data store.

Future Enhancements:

- **Multi-Platform Application Support:** Extending the automation engine to support additional job portals such as Naukri.com, Indeed (auto-apply where feasible), Glassdoor, and Wellfound, leveraging the existing modular architecture to add platform-specific adapters without modifying the core pipeline.
- **Machine Learning-Based Confidence Scoring:** Replacing the current heuristic scoring algorithm with a supervised learning model trained on the user’s historical accept and reject decisions, enabling personalised and continuously improving job suitability predictions.
- **Resume Auto-Generation:** Integrating large language models to dynamically generate tailored resumes for each application based on the job description, the candidate’s master resume, and learned preferences from previous successful applications.
- **Interactive Analytics Dashboard:** Developing a comprehensive web-based dashboard using modern frontend frameworks that provides real-time visualisations of application statistics, funnel analysis, response rates, confidence score distributions, and historical trends.
- **Interview Scheduling Integration:** Connecting with calendar APIs (Google Calendar, Outlook) to automatically create interview events when the Gmail syn-

chronization module detects interview scheduling emails, with automated conflict detection and response suggestions.

Naukri_Guru represents a meaningful advancement toward intelligent job search automation, demonstrating that the integration of modern AI capabilities with robust software engineering practices can substantially reduce the burden of job applications while maintaining data integrity, user privacy, and platform compliance. The open-source nature of the project ensures that the techniques and architectures developed here can be extended and adapted by the broader developer community.

Bibliography

- [1] LinkedIn Corporation. (2024). *About LinkedIn*. <https://about.linkedin.com/>. Accessed: May 2026.
- [2] LazyApply Inc. (2023). *LazyApply – Automate Your Job Applications*. <https://lazyapply.com/>. Accessed: April 2026.
- [3] Woelkl, N. and Contributors. (2023). *EasyApplyBot: LinkedIn Easy Apply Automation*. GitHub Repository. https://github.com/nichrdev/Auto_job_applier_linkedin. Accessed: March 2026.
- [4] JobScan Inc. (2024). *JobScan – Optimize Your Resume for ATS*. <https://www.jobscan.co/>. Accessed: April 2026.
- [5] SeleniumHQ. (2024). *Selenium WebDriver Documentation*. <https://www.selenium.dev/documentation/webdriver/>. Accessed: May 2026.
- [6] Google Chrome Team. (2024). *Puppeteer – Headless Chrome Node.js API*. <https://pptr.dev/>. Accessed: April 2026.
- [7] ultrafunkamsterdam. (2024). *undetected-chromedriver: Custom Selenium ChromeDriver*. GitHub Repository. <https://github.com/ultrafunkamsterdam/undetected-chromedriver>. Accessed: May 2026.
- [8] OpenAI. (2024). *OpenAI API Reference – Chat Completions*. <https://platform.openai.com/docs/api-reference/chat>. Accessed: May 2026.
- [9] Affinda Pty Ltd. (2024). *Affinda Resume Parser – AI-Powered Document Intelligence*. <https://www.affinda.com/resume-parser>. Accessed: April 2026.
- [10] Boni García, S. (2024). *WebDriverManager for Python*. GitHub Repository. https://github.com/SergeyPirogov/webdriver_manager. Accessed: May 2026.
- [11] Google DeepMind. (2024). *Gemini API – Google AI for Developers*. <https://ai.google.dev/gemini-api/docs>. Accessed: May 2026.
- [12] DeepSeek AI. (2024). *DeepSeek API Documentation*. <https://platform.deepseek.com/api-docs>. Accessed: April 2026.

- [13] SQLite Consortium. (2024). *SQLite Documentation*. <https://www.sqlite.org/docs.html>. Accessed: May 2026.
- [14] McKinney, W. and the pandas Development Team. (2024). *pandas: Powerful Data Structures for Data Analysis*. <https://pandas.pydata.org/docs/>. Accessed: May 2026.
- [15] Crispin, M. (2003). *Internet Message Access Protocol – Version 4rev1*. RFC 3501, Internet Engineering Task Force. <https://tools.ietf.org/html/rfc3501>.
- [16] Richardson, L. (2024). *Beautiful Soup Documentation*. <https://www.crummy.com/software/BeautifulSoup/bs4/doc/>. Accessed: May 2026.
- [17] Vastel, A., Laperdrix, P., Rudametkin, W., and Rouvoy, R. (2018). *FP-STALKER: Tracking Browser Fingerprint Evolutions*. In Proceedings of the IEEE Symposium on Security and Privacy (S&P), pp. 728–741.
- [18] Khandait, S.P. and Thool, R.C. (2021). *A Survey on Web Scraping and Automation Techniques for Data Extraction from Dynamic Websites*. International Journal of Computer Applications, 175(15), pp. 32–38.